

11주차 예습과제

📖 자연어처리와 컴퓨터비전 심층학습6장(Word2Vec~)

Word2Vec

- 2013년 구글에서 공개한 임베딩 모델
- 단어 간 유사성 측정하기 위해 **분포 가설(distributional hypothesis)** 기반으로 개발

분포 가설(distributional hypothesis)

- 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미 가질 가능성 높다
- 단어 간 **동시 발생(co-occurrence)** 확률 분포 이용해 단어 간 유사성 측정
 - 단어의 **분산 표현(Distributed Representation)** 학습
 - 분산 표현: 단어를 고차원 벡터 공간에 매핑해 단어의 의미 담는 것
- 단어의 의미는 문맥상 분포적 특성 통해 나타남
 - ⇒ 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치 가짐
- 빈도 기반의 벡터화 기법에서 발생했던 단어의 의미 정보를 저장하지 못하는 한계 극복
- 대량의 텍스트 데이터에서 단어 간의 관계 파악하고 벡터 공간상에서 유사한 단어를 군집화해 단어의 의미 정보 효과적으로 표현
- 분산 표현 방식은 다양한 자연어 처리 작업, 다운스트림 작업에서 높은 성능

단어 벡터화

희소 표현(sparse representation)

- 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법
- 대부분의 벡터 요소가 0으로 표현됨
- 단어 사전의 크기 커지면 벡터의 크기도 커짐 → 공간적 낭비
- 단어 간 유사성을 반영 X, 벡터 간 유사성 계산에 많은 비용

밀집 표현(dense representation)

- Word2Vec

- 단어를 고정된 크기의 실수 벡터로 표현 → 단어 사전의 크기 커져도 벡터의 크기가 커지지 않음
- 벡터 공간상에서 단어 간의 거리 효과적으로 계산
- 대부분의 벡터가 0이 아닌 실수이므로 효율적 공간 활용 가능
- 학습을 통해 단어를 벡터화하기 때문에 단어의 의미 비교할 수 있음
 - **단어 임베딩 벡터(Word Embedding Vector):** 밀집 표현된 벡터
 - Word2Vec은 대표적인 단어 임베딩 기법
 - 밀집 표현을 위해 CBoW, Skip-gram이라는 두 가지 방법 사용

표 6.3 단어의 희소 표현

소	0	1	0	0	0
일고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
일고	-0.2893	0.3814	...	-0.1492	-0.2814

외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

CBoW(Continuous Bag of Words)

- 주변에 있는 단어 가지고 중간에 있는 단어 예측하는 방법
- **중심 단어(Center Word):** 예측해야 할 단어
- **주변 단어(Context Word):** 예측에 사용되는 단어
- **윈도(Window):** 중심 단어를 맞추기 위해 고려할 주변 단어의 범위
 - 윈도를 활용해 주어진 하나의 문장에서 첫 번째 단어부터 중심 단어로 하여서 마지막 단어까지 학습
- **슬라이딩 윈도(Sliding Window):** 윈도를 이동해 가며 학습하는 방법

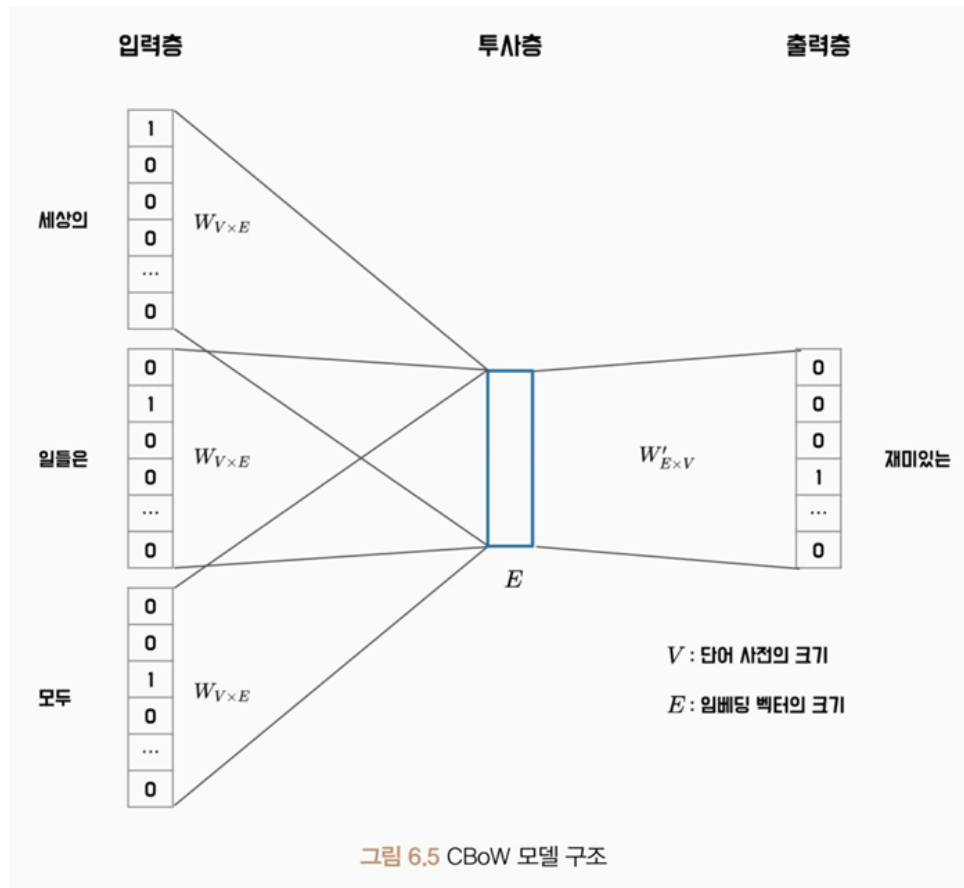
- CBoW는 슬라이딩 윈도우를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어를 학습

Ex) 하나의 입력 문장에서 윈도우 크기가 2일 때 학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

- 학습 데이터는 (주변 단어 | 중심 단어)로 구성
→ 대량의 말뭉치에서 효율적으로 단어의 분산 표현 학습
- 얻어진 학습 데이터는 인공 신경망을 학습하는 데 사용



- CBoW 모델
 - 입력값: 각 입력 단어의 원-핫 벡터
 - 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터를 예측
 - 입력 단어는 원-핫 벡터로 표현되 **투사층(Projection Layer)**에 입력됨
 - 투사층은 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 **순람표 (Lookup table, LUT)** 구조
 - 투사층 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환됨
 그림에서 입력된 세 단어의 임베딩 벡터를 $V_{\text{세상의}}$, $V_{\text{일들은}}$, $V_{\text{모두}}$ 라 가정하면 이 벡터의 평균값을 계산
 - 계산된 평균 벡터를 가중치 행렬 $W'_{E \times V}$ 와 곱해 V 크기의 벡터를 얻음
 - 얻은 벡터에 소프트맥스 함수 이용해서 중심 단어 예측

Skip-gram

- 중심 단어를 입력으로 받아 주변 단어를 예측하는 모델

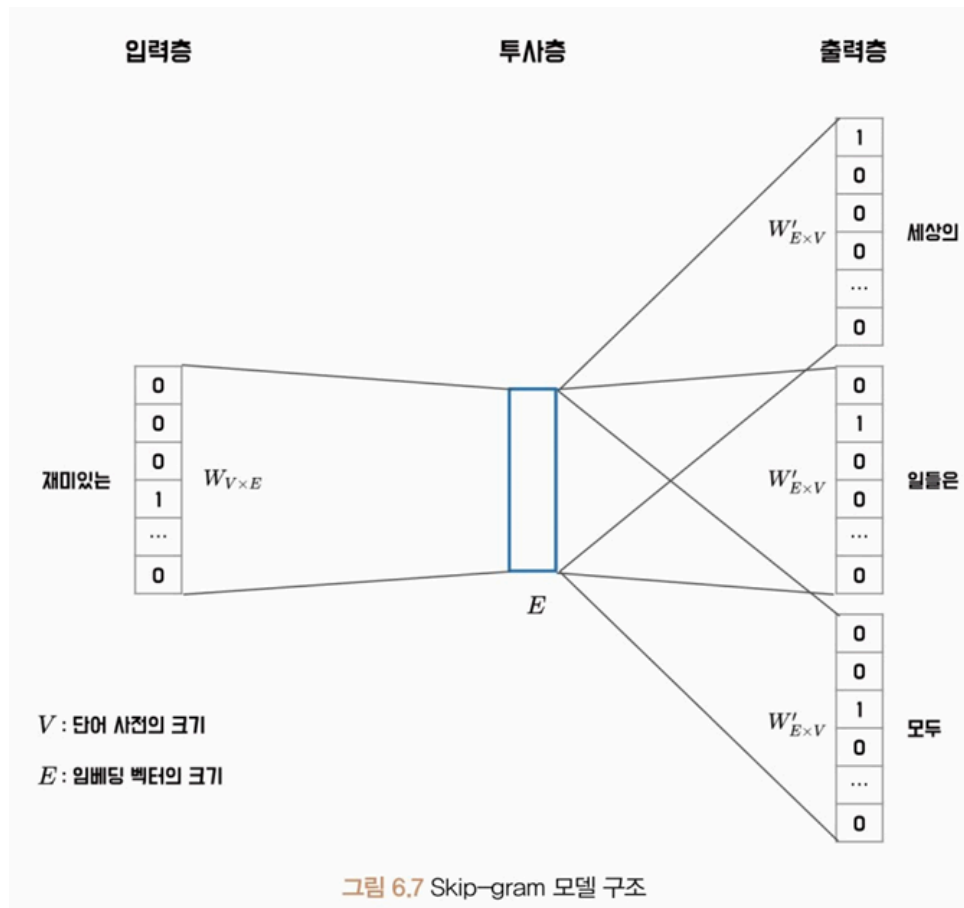
- 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋 만들기
 - 이때 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델을 학습

Ex) Skip-gram에서 윈도우 크기가 2일 때 학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

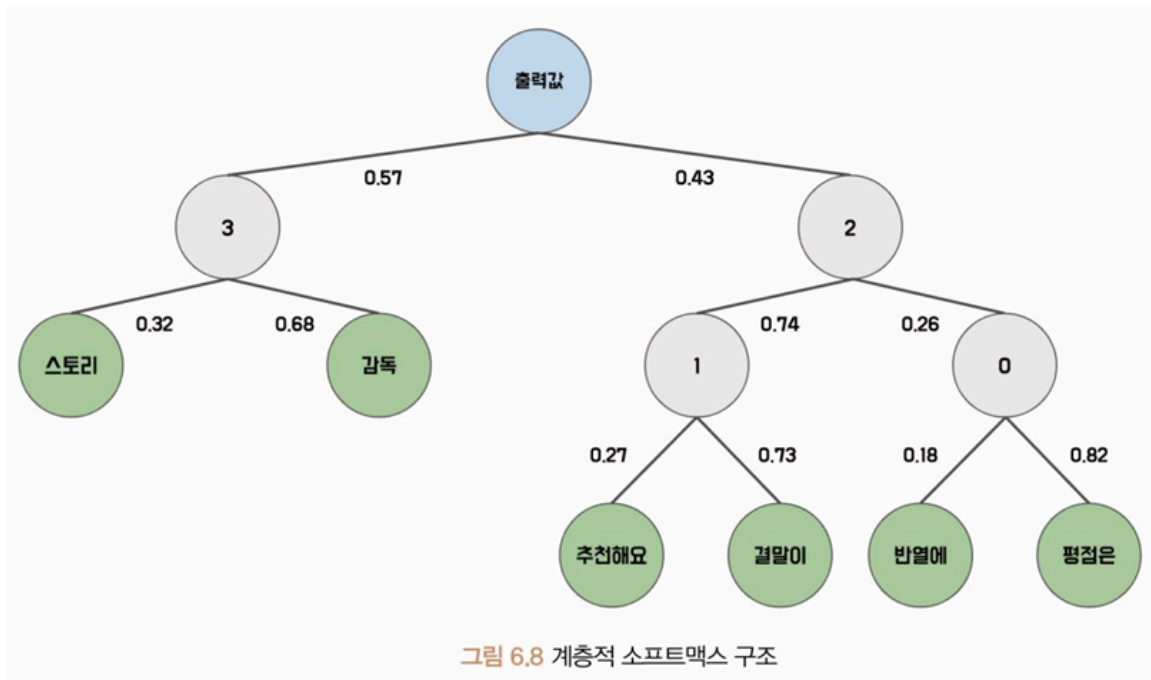
- Skip-gram과 CBoW의 차이: 학습 데이터의 구성 방식
 - CBoW: 하나의 윈도우에서 하나의 학습 데이터
 - Skip-gram: 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터
 - Skip-gram은 하나의 중심 단어를 통해 여러 개의 주변 단어 예측하므로 **더 많은 학습 데이터셋** 추출 가능, 일반적으로 CBoW보다 더 뛰어난 성능
 - Skip-gram은 비교적 드물게 등장하는 단어를 더 잘 학습 가능, 단어 벡터 공간에서 더 유의미한 거리 관계 형성 가능



- Skip-gram 모델
 - 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져옴
 - 입력 단어의 임베딩과 $W'_{E \times V}$ 가중치와의 곱셈을 통해 V 크기의 벡터 얻음
 - 얻은 벡터에 소프트맥스 연산 취해 주변 단어 예측
- 소프트맥스 연산은 모든 단어를 대상으로 내적 연산 수행
 - 말뭉치의 크기 커지면 단어 사전의 크기도 커지므로 대량의 말뭉치 통해 Word2Vec 모델 학습 시 학습 속도가 느려지는 단점
 - 계층적 소프트맥스, 네거티브 샘플링 기법 적용해 학습 속도가 느려지는 문제 완화

계층적 소프트맥스(Hierarchical Softmax)

- 출력층을 이진 트리(Binary tree) 구조로 표현해 연산 수행
- 자주 등장하는 단어일수록 트리의 상위 노드, 드물게 등장하는 단어일수록 하위 노드에 배치됨
- 일반적 소프트맥스 연산에 비해 빠른 속도와 효율성



- 계층적 소프트맥스의 구조
 - 각 노드는 학습이 가능한 벡터를 가짐
 - 입력값은 해당 노드의 벡터와 내적값 계산한 후 시그모이드 함수를 통해 확률을 계산
 - **잎 노드(Leaf Node)**: 가장 깊은 노드, 각 단어를 의미함
 - 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 함
 - 각 단어의 확률은 경로 노드의 확률 곱해 구할 수 있음

Ex) '추천해요' → $0.43 \times 0.74 \times 0.27 = 0.085914$ 의 확률, 학습 시 1, 2번 노드의 벡터만 최적화하면 됨
- 일반적인 소프트맥스 연산은 $O(V)$ 의 시간 복잡도 갖지만, 계층적 소프트맥스는 $O(\log_2 V)$ 의 시간 복잡도 가짐 (V = 단어 사전의 크기)

네거티브 샘플링(Negative Sampling)

- Word2Vec 모델에서 사용되는 확률적인 샘플링 기법
 - 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용
 - 학습 원도 내에 등장하지 않는 단어를 n개 추출하여 정답 단어와 함께 소프트맥스 연산을 수행
- ⇒ 전체 단어의 확률 계산할 필요 없이 모델을 효율적으로 학습 가능

- 추출할 단어 n 은 일반적으로 5~20개 사용

수식 6.9 네거티브 샘플링의 추출 확률

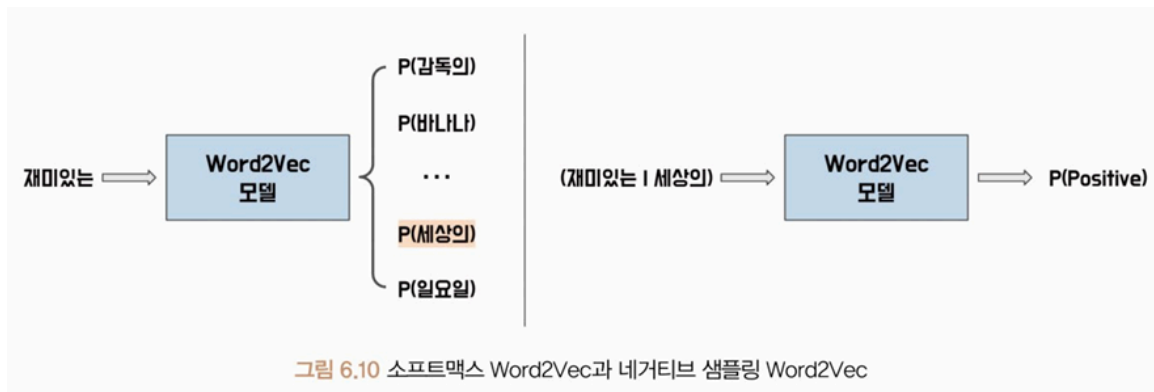
$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- $f(w_i)$: 각 단어 w_i 의 출현 빈도수
Ex) 말뭉치 내에 단어 '추천해요' 100번 등장, 전체 단어의 빈도 2000 → $f(\text{추천해요}) = 100/2000 = 0.05$
- $P(w_i)$: 단어 w_i 가 네거티브 샘플로 추출될 확률
 - 출현 빈도수에 0.75제곱한 값을 정규화 상수로 사용
- 네거티브 샘플링에서는 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 네거티브 샘플링으로 생성된 단어 쌍인지 이진분류
→ 로지스틱 회귀 모델 사용, 학습 과정에서 추출할 단어의 확률 분포 구하기 위해 먼저 각 단어에 대한 가중치를 학습
- 일반 Skip-gram 모델과 네거티브 샘플링 Word2Vec 모델의 훈련 데이터 차이



- 네거티브 샘플링 Word2Vec 모델은 실제 데이터에서 추출된 단어 쌍은 1, 네거티브 샘플링을 통해 추출된 가짜 단어 쌍은 0으로 레이블링
⇒ 다중 분류에서 이진 분류로 학습 목적 바뀜

- 일반적 Word2Vec 모델과 네거티브 샘플링 Word2Vec 모델의 출력 구조 차이



- 네거티브 샘플링 모델에서는 입력 단어의 임베딩과 해당 단어가 맞는지 여부 나타내는 레이블(1 또는 0) 가져와 내적 연산 수행
- 내적 연산 통해 얻은 값은 시그모이드 함수 통해 확률값으로 변환
- 레이블이 1인 경우 해당 확률값이 높아지도록, 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화 됨

모델 실습: Skip-gram

- Word2Vec 모델은 학습할 단어의 수를 V 로, 임베딩 차원을 E 로 설정해 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬을 최적화하며 학습
- $W_{V \times E}$ 행렬은 **룩업(Lookup)** 연산을 수행 → **임베딩(Embedding)** 클래스 사용해 구현
 - 임베딩 클래스는 범주형 변수와 같은 이산 변수를 연속적인 벡터 형태로 변환해 사용 가능
 - 연속적인 벡터 표현은 단어의 의미와 관련된 정보 포착, 이를 기반으로 단어 간의 유사도를 계산
 - 이때 이산 변수의 값을 연속적인 벡터로 변환하는 과정을 룩업이라 함



```
#임베딩 클래스
embedding = torch.nn.Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None,
    norm_type=2.0
)
```

- **임베딩 수(`num_embeddings`)**: 이산 변수의 개수, 단어 사전의 크기 의미
 - **임베딩 차원(`embedding_dim`)**: 임베딩 벡터의 차원 수, 임베딩 벡터의 크기 의미
 - **패딩 인덱스(`padding_idx`)**
 - 패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정
 - 병렬 처리를 위해 입력 배치의 문장 길이가 동일해야 함 → 입력 문장들을 일정한 길이로 맞추는 역할
 - 임베딩 수보다 작아야 하며 패딩 인덱스의 벡터값은 모델 학습 시 최적화되지 않음
 - **노름 타입(`norm_type`)**: 임베딩 벡터의 크기를 제한하는 방법 선택
 - 기본값은 2 → L2 정규화 방식 사용한다는 의미 (1로 설정 시 L1 정규화 사용)
 - **최대 노름(`max_norm`)**: 임베딩 벡터의 최대 크기 지정
 - 각 임베딩 벡터의 크기가 최대 노름 이상이면 임베딩 벡터를 최대 노름 크기로 잘라내고 크기 감소시킴
-
- 파이토치의 임베딩 클래스로 룩업 계층 구현, 선형 변환 클래스로 가중치 행렬 구현



```
#기본 Skip-gram 클래스
from torch import nn

class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim
        )
        self.linear = nn.Linear(
            in_features=embedding_dim,
            out_features=vocab_size
        )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output
```

- 기본 형식의 Skip-gram 클래스
 - 단순히 입력 단어와 주변 단어를 룩업 테이블에서 가져와 내적을 계산 후, 손실 함수를 통해 예측 오차 최소화하는 방식으로 학습



#영화 리뷰 데이터셋 전처리

```
import pandas as pd
```

```
from Korpora import Korpora
```

```
from konlpy.tag import Okt
```

```
corpus = Korpora.load("nsmc")
```

```
corpus = pd.DataFrame(corpus.test)
```

```
tokenizer = Okt()
```

```
tokens = [tokenizer.morphs(review) for review in corpus.text]
```

```
print(tokens[:3])
```

```
[['꾼', 'ㅋ'], ['GDNTOPCLASSINTHECLUB'], ['뭐', '야', '이', '평점', '들', '은', '.....', '나쁘진', '않지만', '10', '점', '짜', '리', '는', '더', '더욱', '아니잖아']]
```

- 코포라 라이브러리의 네이버 영화 리뷰 감정 분석 데이터셋
- `corpus.test` 로 테스트 세트 불러오기
- 데이터셋을 `Okt` 토크나이저 사용해 형태소 추출



```
#단어 사전 구축
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

vocab = build_vocab(corpus=tokens, n_vocab=5000, special_tokens=["<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

```
['<unk>', '.', '이', '영화', '의', '..', '가', '에', '...', '을']
5001
```

- **Ok** 토큰라이저를 통해 토큰화된 데이터 활용해 `build_vocab` 함수로 단어 사전 구축
 - `n_vocab` : 구축할 단어 사전의 크기 의미
 - 문서 내에 `n_vocab` 보다 많은 종류의 토큰 있다면 가장 많이 등장한 토큰 순서로 사전을 구축
 - `special_tokens` : 특별한 의미 갖는 토큰 의미
 - `<unk>` 토큰은 OOV에 대응하기 위한 토큰, 단어 사전 내에 없는 모든 단어들은 `<unk>` 토큰으로 대체됨
- 예제는 단어 사전의 최대 길이를 5000으로 설정
 - **특수 토큰(Special token)**이 현재 1개이므로 단어 사전의 크기는 5,001개



#Skip-gram의 단어 쌍 추출

```
def get_word_pairs(tokens, window_size):
    pairs = []
    for sentence in tokens:
        sentence_length = len(sentence)
        for idx, center_word in enumerate(sentence):
            window_start = max(0, idx - window_size)
            window_end = min(sentence_length, idx + window_size + 1)
            center_word = sentence[idx]
            context_words = sentence[window_start:idx] + sentence[idx+1:window_end]
            for context_word in context_words:
                pairs.append([center_word, context_word])
    return pairs

word_pairs = get_word_pairs(tokens, window_size=2)
print(word_pairs[:5])
```

[['굳', '='], ['=', '굳'], ['뭐', '야'], ['뭐', '이'], ['야', '뭐']]

- 중심 단어의 앞, 뒤 두 개 단어를 윈도우로 하여 단어 쌍 추출 방식 보여줌
- `get_word_pairs` 함수
 - 토큰을 입력받아 Skip-gram 모델의 각 입력 데이터로 사용할 수 있게 전처리
 - `window_size` : 주변 단어를 몇 개까지 고려할 것인지 설정
 - 각 문장에서는 중심 단어와 주변 단어를 고려하여 쌍을 생성
 - `idx` : 현재 단어의 인덱스
 - `center_word` : 중심 단어
 - `window_start`, `window_end` : 현재 단어에서 얼마나 멀리 떨어진 주변 단어를 고려할 것인지 결정
 - 이때 문장의 경계를 넘어가는 경우 없게 조정
- 출력 결과를 보면 각 단어 쌍은 [중심 단어, 주변 단어]로 구성됨



- 임베딩 층은 단어의 인덱스를 입력으로 받기 때문에 단어 쌍을 인덱스 쌍으로 변환해야 함

#인덱스 쌍 변환

```
def get_index_pairs(word_pairs, token_to_id):
    pairs = []
    unk_index = token_to_id["<unk>"]
    for word_pair in word_pairs:
        center_word, context_word = word_pair
        center_index = token_to_id.get(center_word, unk_index)
        context_index = token_to_id.get(context_word, unk_index)
        pairs.append([center_index, context_index])
    return pairs
```

```
index_pairs = get_index_pairs(word_pairs, token_to_id)
print(index_pairs[:5])
```

```
[[595, 100], [100, 595], [77, 176], [77, 2], [176, 77]]
```

- `get_index_pairs` 함수
 - `get_word_pairs` 함수에서 생성된 단어 쌍을 토큰 인덱스 쌍으로 변환
 - `word_pairs` , 단어와 단어에 해당하는 ID를 매핑한 딕셔너리인 `token_to_id` 로 인덱스 쌍을 생성
 - 딕셔너리의 `get` 메서드로 토큰이 단어 사전 내에 있으면 해당 토큰의 인덱스를 반환, 없으면 `<unk>` 토큰의 인덱스를 반환
- 생성된 인덱스 쌍은 Skip-gram 모델의 입력 데이터로 사용



- 학습을 위해 텐서 형식으로 변환

```
#데이터로더 적용
import torch
from torch.utils.data import TensorDataset, DataLoader

index_pairs = torch.tensor(index_pairs)
center_indexes = index_pairs[:, 0]
content_indexes = index_pairs[:, 1]

dataset = TensorDataset(center_indexes, content_indexes)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

- `index_pairs`
 - `get_index_pairs` 함수에서 생성된 중심 단어와 주변 단어 토큰의 인덱스 쌍으로 이루어진 리스트
 - 텐서 형식으로 변환 → [N, 2]의 구조 가지므로 중심 단어와 주변 단어로 나눠 데이터셋으로 변환



```
#Skip-gram 모델 준비 작업
from torch import optim

device = "cuda" if torch.cuda.is_available() else "cpu"
word2vec = VanillaSkipgram(vocab_size=len(token_to_id), embedding_dim=128).to(device)
criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(word2vec.parameters(), lr=0.1)
```

- `VanillaSkipgram` 클래스의 단어 사전 크기(`vocab_size`)에 전체 단어 집합의 크기를 전달, 임베딩 크기(`embedding_dim`)는 128로 할당
- 손실함수는 단어 사전 크기만큼 클래스가 있는 분류 문제이므로 교차 엔트로피 사용
 - 크로스 엔트로피는 내부적으로 소프트맥스 연산 수행 → 신경망의 출력값을 후처리 없이 활용할 수 있음



```
#모델 학습
for epoch in range(10):
    cost = 0.0

    for input_ids, target_ids in dataloader:
        input_ids = input_ids.to(device)
        target_ids = target_ids.to(device)

        logits = word2vec(input_ids)
        loss = criterion(logits, target_ids)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    cost += loss

cost = cost / len(dataloader)
print(f"Epoch : {epoch+1:4d}, Cost : {cost:.3f}")
```

```
Epoch :    1, Cost : 5.802
Epoch :    2, Cost : 5.794
Epoch :    3, Cost : 5.786
Epoch :    4, Cost : 5.778
Epoch :    5, Cost : 5.771
Epoch :    6, Cost : 5.764
Epoch :    7, Cost : 5.757
Epoch :    8, Cost : 5.751
Epoch :    9, Cost : 5.745
Epoch :   10, Cost : 5.739
```

- 모델 학습이 완료되면 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬 중 하나의 행렬을 선택해 임베딩 값을 추출



- 파이토치의 임베딩 층으로 구현된 $W_{V \times E}$ 행렬에서 임베딩 값을 추출

```
#임베딩 값 추출
token_to_embedding = dict()
embedding_matrix = word2vec.embedding.weight.detach().cpu().numpy()

for word, embedding in zip(vocab, embedding_matrix):
    token_to_embedding[word] = embedding

index = 30
token = vocab[30]
token_embedding = token_to_embedding[token]
print(token)
print(token_embedding)
```

```
연기
[-0.55641204 -0.4307796  0.43644798  0.31433013 -0.2085117  0.89748746
 1.0048636  1.1549672 -0.35622403  0.9792936 -0.9391772  0.55544186
-0.8535791 -0.11812089 -0.31378892 -0.9531512  0.73158324 -0.9413806
 0.9633818  0.19018474  1.0617411 -0.40510914 -1.1060432  0.5973689
-0.6346264 -0.6572101  1.2999566  0.00344572  0.9816693  1.3209983
-0.44959044 0.22650322 -0.5794389 -0.31680617 -2.0614777 -0.09265196
 0.433568 -0.30555052  0.58536965 -1.2211871 -1.6738548 -0.1312986
 1.852386 -1.3280761 -1.3050271  0.7531579 -1.6161251  0.64032984]
```

- Word2Vec 모델의 임베딩 행렬을 이용해 각 단어의 임베딩 값을 매핑, 인덱스 30의 단어와 임베딩 값을 출력
→ 이 임베딩 값으로 단어 간의 유사도를 확인 가능

• 코사인 유사도(Cosine Similarity)

- 임베딩의 유사도를 측정할 때 가장 일반적으로 사용되는 방법
- 두 벡터 간의 각도를 이용하여 유사도를 계산
- 두 벡터가 유사할수록 값이 1에, 다를수록 0에 가까워짐

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- 두 벡터의 내적을 벡터의 크기(유클리드 노름)의 곱으로 나누어 계산
- 코사인 유사도는 임베딩 공간에서 단어 간의 유사도를 특정하는 데 유용



- 코사인 유사도로 특정 단어와 가장 유사한 단어를 출력

```
#단어 임베딩 유사도 계산
import numpy as np
from numpy.linalg import norm

def cosine_similarity(a, b):
    cosine = np.dot(b, a)/(norm(b, axis=1)*norm(a))
    return cosine

def top_n_index(cosine_matrix, n):
    closest_indexes = cosine_matrix.argsort()[::-1]
    top_n = closest_indexes[1 : n + 1]
    return top_n

cosine_matrix = cosine_similarity(token_embedding, embedding_matrix)
top_n = top_n_index(cosine_matrix, n=5)

print(f"{token}와 가장 유사한 5 개 단어")
for index in top_n:
    print(f"{id_to_token[index]} - 유사도: {cosine_matrix[index]:.4f}")
```

```
연기와 가장 유사한 5 개 단어
정석 - 유사도: 0.3900
찍고 - 유사도: 0.3146
배역 - 유사도: 0.3088
캐릭터 - 유사도: 0.2831
송강호 - 유사도: 0.2802
```

- 넘파이의 선형대수 라이브러리를 활용해 벡터와 벡터의 코사인 유사도나 벡터와 행렬의 코사인 유사도를 쉽게 계산 가능
- `cosine_similarity` 함수
 - 입력 단어와 단어 사전 내의 모든 단어와의 코사인 유사도를 계산
 - `a` 매개변수는 임베딩 토큰, `b` 매개변수는 임베딩 행렬을 의미
 - 임베딩 행렬은 [5001, 128]의 구조 → 노름 계산할 때 `axis=1` 방향으로 계산
- `top_n_index` 함수
 - 유사도 행렬을 내림차순으로 정렬해 어떤 단어가 가장 가까운 단어인지 반환
 - 입력 단어도 단어 자신에 포함되므로 입력 단어 자신이 가장 가까운 단어가 되기 때문에 두 번째로 가까운 단어부터 계산해 반환

모델 실습: Gensim

- 계층적 소프트맥스나 네거티브 샘플링 같은 기법 사용하면 더 효율적 학습 가능
- **젠심(Gensim)** 라이브러리
 - Word2Vec과 같은 자연어 처리 모델 쉽게 구성 가능
 - 대용량 텍스트 데이터의 처리를 위한 메모리 효율적인 방법 제공, 대규모 데이터셋에서도 효과적으로 모델 학습
 - 학습된 모델 저장하여 관리할 수 있고, 비슷한 단어 찾기 등 유사도와 관련된 기능도 제공
- 젠심 라이브러리는 사이썬을 이용해 병렬 처리나 네거티브 샘플링 등을 적용
 - 사이썬은 C++ 기반의 확장 모듈을 파이썬 모듈로 컴파일하는 기능을 제공
 - 젠심으로 모델을 구성하면 파이토치를 이용한 학습보다 훨씬 더 빠른 속도로 학습 가능



- 젠심 라이브러리의 `Word2Vec` 클래스는 네거티브 샘플링 등에 필요한 여러 하이퍼파라미터를 받아 쉽고 빠르게 모델을 학습 가능

```
#Word2Vec 클래스
word2vec = gensim.models.Word2Vec(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window=5,
    min_count=5,
    workers=3,
    sg=0,
    hs=0,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000
)
```

- **입력 문장(`sentences`)**
 - 모델의 학습 데이터를 나타내며 토큰 리스트로 표현됨
 - 구성된 학습 데이터는 학습 문장들의 리스트로 이루어짐
- **말뭉치 파일(`corpus_file`)**
 - 학습 데이터를 파일로 입력할 때 파일의 경로를 의미
 - 입력 문장 대신 사용할 수 있음
- **벡터 크기(`vector_size`)**
 - 학습할 임베딩 벡터의 크기를 의미, 임베딩 차원 수 설정
- **학습률(`alpha`)**
 - Word2Vec 모델의 학습률을 의미
- **윈도(`window`)**
 - 학습 데이터를 생성할 윈도의 크기 의미
Ex) 윈도가 3이라면 중심 단어로부터 3만큼 떨어진 단어까지 주변 단어로 고려해 데이터 생성
- **최소 빈도(`min_count`)**

- 학습에 사용할 단어의 최소 빈도 의미
- 말뭉치 내에 최소 빈도만큼 등장하지 않은 단어는 학습에 사용되지 않음
- **최대 최종 단어 사전(`max_final_vocab`)**
 - 단어 사전의 최대 크기 의미
 - 최소 빈도를 충족하는 단어가 최대 최종 단어 사전보다 많으면 자주 등장한 단어 순으로 단어 사전을 구축
- **작업자 수(`workers`)**
 - 빠른 학습을 위해 병렬 학습할 스레드의 수 의미
- **Skip-gram(`sg`)**
 - Skip-gram 모델 사용 여부 설정
 - 1이면 Skip-gram, 0이면 CBoW 모델 사용
- **계층적 소프트맥스(`hs`)**
 - 계층적 소프트맥스 사용 여부 설정
 - 1이면 사용, 0이면 사용 X
- **CBoW 평균화(`cbow_mean`)**
 - CBoW 모델로 구성할 때 사용되는 하이퍼파라미터
 - 중심 단어와 주변 단어를 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부 설정
 - 1이면 평균화, 0이면 평균화 하지 않고 합산
- **네거티브(`negative`)**
 - 네거티브 샘플링의 단어 수 의미, 0이라면 네거티브 샘플링 사용 X
- **네거티브 지수(`ns_exponent`)**
 - 네거티브 샘플링 확률의 지수 의미
- **에폭(`epoch`)**
 - 학습 에폭 수 의미
- **배치 단어 수(`batch_words`)**
 - 몇 개의 단어로 학습 배치를 구성할 지 결정
 - 컴퓨팅 자원이 모자라다면 배치 단어 수 낮추어 학습 진행



```
#Word2Vec 모델 학습
from gensim.models import Word2Vec

word2vec = Word2Vec(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=1,
    sg=1,
    epochs=3,
    max_final_vocab=10000
)

# word2vec.save("../models/word2vec.model")
# word2vec = Word2Vec.load("../models/word2vec.model")
```

- Word2Vec 클래스 활용해 모델 학습
 - 입력 문장은 앞선 예제에서 토큰화한 값 입력, 임베딩 벡터 크기 128, 윈도우 크기 5로 설정
 - 최소 빈도 고려 X, 최대 단어 사전의 크기는 10,000으로 설정
 - 빠른 학습을 위해 네거티브 샘플링 사용, 에폭 3으로 설정해 모델 학습
- 앞선 `VanillaSkipgram` 클래스와 비교하면 매우 빠르게 학습되는 것 알 수 있음
- `word2vec` 인스턴스는 파이토치의 저장 메서드와 동일한 방법으로 저장 가능



#임베딩 추출 및 유사도 계산

```
word = "연기"
print(word2vec.wv[word])
print(word2vec.wv.most_similar(word, topn=5))
print(word2vec.wv.similarity(w1=word, w2="연기력"))
```

```
[-2.20186085e-01 -2.94060498e-01 1.27066895e-01 7.67192125e-01
-2.56483015e-02 2.18421310e-01 -5.26300352e-03 -1.12382896e-01
-5.98965943e-01 4.10539567e-01 -1.04715139e-01 -2.82925308e-01]
```

```
[('연기력', 0.7793591022491455), ('캐스팅', 0.7427349090576172), ('연기자', 0.7287553548812866), ('여배우', 0.7200956344604492), ('목소리', 0.713671088218689)]
0.779359
```

- `word2vec` 인스턴스의 `wv` 속성은 학습된 단어 벡터 모델을 포함한 `Word2VecKeyedVectors` 객체를 반환
- `Word2VecKeyedVectors` 객체
 - 단어 벡터 검색과 유사도 계산 등의 작업 수행에 사용
 - 주어진 단어에 대해 가장 유사한 단어를 찾는 `most_similar` 메서드와 두 단어 간의 유사도를 계산하는 `similarity` 메서드 제공
- Word2Vec은 분포 가설을 통해 쉽고 빠르게 단어의 임베딩 학습할 수 있지만, 단어의 형태학적 특징을 반영하지 못한다는 한계

Ex) 교착어로서 한국어는 어근, 접사, 조사 등으로 이루어지는 규칙 가짐

→ Word2Vec 모델에서는 이러한 구조적 특징 제대로 학습하기 어려움

⇒ 제한된 단어 사전에서 많은 OOV 발생시키는 원인

fastText

- **fastText**
 - 2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델
 - 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘
 - 단어와 문장을 벡터로 변환하는 기술을 기반으로 하여 이를 통해 머신러닝 알고리즘이 텍스트 데이터를 분석하고 이해하는 데 사용
- 벡터화 기술은 Word2Vec과 유사하지만 fastText는 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능 제공
 - 하위 단어를 고려하기 위해 N-gram 사용해 단어 분해하고 벡터화

- 단어의 벡터화를 위해 〈, 〉 와 같은 특수 기호를 사용해 단어의 시작과 끝 나타냄 → 단어의 하위 문자열을 고려하는 데 중요한 역할
- 기호가 추가된 단어는 N-gram을 사용하여 **하위 단어 집합(Subword set)**으로 분해됨
Ex) '서울특별시'를 바이그램으로 분해 → '서울', '울특', '특별', '별시'
- 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함됨
- 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 가짐
- 이러한 하위 단어 벡터들은 단어의 벡터 표현을 구성, 이를 사용해 자연어 처리 작업을 수행
- fastText 모델이 입력 단어 처리하는 방법

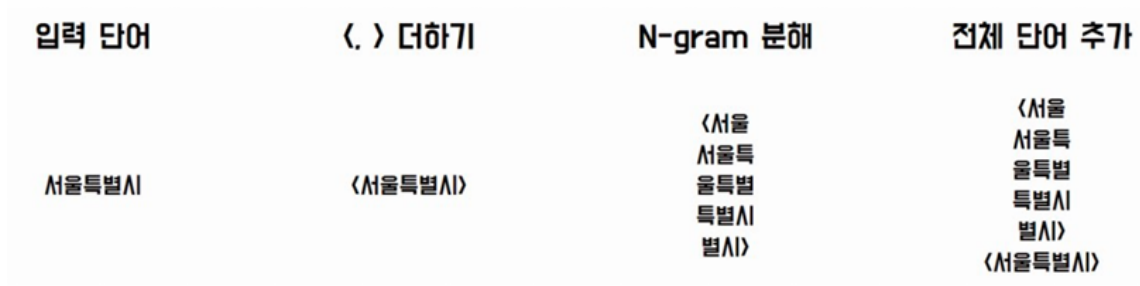


그림 6.11 fastText의 하위 단어 집합

1. 토큰의 양 끝에 '〈'와 '〉'를 붙여 토큰의 시작과 끝을 인식할 수 있게 한다.
 2. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해한다. 트라이그램을 사용한다면, '서울특별시'는 [〈서울, 서울특, 울특별, 특별시, 별시〉]로 분해된다.
 3. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함한다. 이렇게 하위 단어 집합이 만들어지면, 각 하위 단어는 고유한 벡터값을 갖는다.
- 각 하위 단어의 임베딩 벡터를 구하고, 이를 모두 합산하여 입력 단어의 최종 임베딩 벡터를 계산
- fastText는 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 벡터를 구성함으로써 단어의 부분 문자열을 고려하는 유연하고 정확한 하위 단어 집합을 생성
⇒ 같은 하위 단어를 공유하는 단어끼리는 정보를 공유해 학습할 수 있음
→ 비슷한 단어끼리는 비슷한 임베딩 벡터 갖게 되어 단어 간 유사도 높일 수 있음
 - OOV 단어도 하위 단어로 나누어 임베딩을 계산할 수 있게 됨
 - 이렇게 하위 단어 기반의 임베딩 방법 사용하면 말뭉치에 등장하지 않은 단어라도 유사한 하위 단어 가지고 있으면 유사한 임베딩 벡터 갖게 됨

모델 실습

- fastText와 Word2Vec 모델 모두 단어 임베딩을 학습하는 데 사용되는 알고리즘
 - 단어를 고정 길이 벡터 형태로 표현하기 위해 분산 표현 학습을 수행, 주변 단어의 정보를 활용하여 단어의 의미를 파악
 - 이를 통해 단어 간의 유사성을 측정, 비슷한 의미를 가진 단어를 유사한 벡터로 표현
 - 유사한 단어들은 공간상 가깝게 임베딩됨
- fastText 모델도 CBoW와 Skip-gram으로 구성되며 네거티브 샘플링 기법을 사용해 학습
 - fastText는 하위 단어로 학습 ($\leftrightarrow$ Word2Vec 모델은 기본 단위)
 - ⇒ Word2Vec 모델보다 입력 단어를 구성하는 데 조금 더 복잡한 과정 필요, 모든 하위 단어 크기를 갖는 임베딩 계층 필요
 - 젠심 라이브러리는 이런 과정을 `FastText` 클래스로 제공



```
fasttext = gensim.models.FastText(  
    sentences=None,  
    corpus_file=None,  
    vector_size=100,  
    alpha=0.025,  
    window=5,  
    min_count=5,  
    workers=3,  
    sg=0,  
    hs=0,  
    cbow_mean=1,  
    negative=5,  
    ns_exponent=0.75,  
    max_final_vocab=None,  
    epochs=5,  
    batch_words=10000,  
    min_n=3,  
    max_n=6  
)
```

- 대부분의 하이퍼파라미터는 **Word2Vec** 클래스의 하이퍼파라미터와 동일
- N-gram 범위를 결정하는 하이퍼파라미터가 추가됨
 - **최소 N(`min_n`)**: 사용할 N-gram의 최소값
 - **최대 N(`max_n`)**: 사용할 N-gram의 최대값

Ex) 최소 N이 2, 최대 N이 4이면 입력 단어를 2-gram, 3-gram, 4-gram
으로 나누어 하위 단어 집합 생성

- KorNLI(Korean Natural Language Inference) 데이터셋
 - 한국어 **자연어 추론(Natural Language Inference, NLI)**을 위한 데이터셋
- 자연어 추론
 - 두 개 이상의 문장이 주어졌을 때, 두 문장 간의 관계를 분류하는 작업
 - 주어진 문장이 함의 관계, 중립 관계, 불일치 관계 중 어느 관계에 해당되는지 분류할 수 있음



```
#KorNLI 데이터셋 천저리
from Korpora import Korpora

corpus = Korpora.load("kornli")
corpus_texts = corpus.get_all_texts() + corpus.get_all_pairs()
tokens = [sentence.split() for sentence in corpus_texts]

print(tokens[:3])
```

[['개념적으로', '크림', '스키밍은', '제품과', '지리라는', '두', '가지', '기본', '차원을', '가지고', '있다.',], ['시즌', '중에', '알고', '있는', '거', '알아?',],

- 코포라 라이브러리의 KorNLI 인스턴스
 - `get_all_texts` 메서드
 - 모든 문장을 튜플 형태로 반환
 - `get_all_pairs` 메서드
 - 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환
- 단어 임베딩 모델을 학습하는 것이 목적이므로 두 문장의 관계를 고려하지 않고 입력 문장과 대응 문장 전체를 이용해 학습을 진행
- fastText 모델은 입력 단어의 구조적 특징을 학습할 수 있음
 - ⇒ 형태소 분석기를 통해 토큰화하지 않고 띄어쓰기를 기준으로 단어를 토큰화해 학습 진행



```
#fastText 모델 실습
from gensim.models import FastText

fastText = FastText(
    sentences=tokens,
    vector_size=128,
    window=5,
    min_count=5,
    sg=1,
    epochs=3,
    min_n=2,
    max_n=6
)

# fastText.save(" .. /models/fastText.model")
# fastText = FastText.load(" .. /models/fastText.model")
```

- 젠심 라이브러리의 `FastText` 클래스 활용한 모델 학습
 - 입력 문장은 앞선 예제에서 토큰화한 값 입력, 임베딩 벡터 크기 128, 윈도우 크기 5
 - 5번 이상 등장한 단어만 사용, 최대 단어 사전의 크기는 20,000, N-gram 범위는 [2, 6]으로 설정
- Word2Vec 모델과 다르게 OOV 단어를 대상으로도 의미 있는 임베딩 추출 가능



- 단어 사전에 없는 단어의 임베딩 추출과 유사도 계산하는 방법

```
#fastText OOV 처리
oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))

False
[('사랑해', 0.9104023575782776), ('사랑', 0.8649243116378784), ('사랑한', 0.8559931516647339), ('사랑해서', 0.8375059366226196), ('사랑해.', 0.8355680704116821)]
```

- `wv.index_to_key` 메서드
 - 학습된 단어 사전을 나타내는 리스트 의미
 - '사랑해요'라는 단어는 단어 사전 리스트 내에 존재하지 않으므로 OOV 토큰으로 처리됨
- fastText는 하위 단어로 나뉘어 있기 때문에 '사랑해요'라는 단어 처리 가능
→ 하위 단어로 나누어 임베딩을 학습하는 fastText의 접근 방식 통해 OOV 문제 효과적으로 해결 가능

순환 신경망

- 순환 신경망(Recurrent Neural Network, RNN)
 - 순서가 있는 **연속적인 데이터(Sequence data)**를 처리하는 데 적합한 구조
 - 각 **시점(Time step)**의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성
- 연속형 데이터
 - 특정 시점에서의 데이터가 이전 시험의 영향을 받는 데이터
 - 자연어 데이터는 연속적인 데이터의 일종
 - 문장 안에서 단어들이 순서대로 등장하므로, 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장의 의미를 형성
 - 금요일이 지나면 _
 - 수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면 _
 - 주어진 문장들에서 이전 단어들의 패턴과 의미를 고려해 다음에 올 단어를 유추할 수 있음

→ 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미를 형성

⇒ 연속형 데이터의 특성 가짐(t 번째 단어는 $t-1$ 번째까지의 단어에 영향을 받아 결정)

- 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 **상관관계 (Correlation)** 존재

→ 이러한 문맥과 상호작용을 모델링해 정확한 의미 파악이 필요

순환 신경망

- 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류
 - 이전에 처리한 데이터를 다음 단계에 활용, 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용
 - 시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터와 같은 도메인에서 사용
 - 길이가 가변적, 순서에 따라 의미 있는 데이터에 적합한 구조
 - 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 **은닉 상태(Hidden state)**의 형태로 저장
 - **셀(Cell)**
 - 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드
 - 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉 상태 h_t 를 계산
- ⇒ 이런 재귀적 특징으로 인해 순환 신경망이라 불림

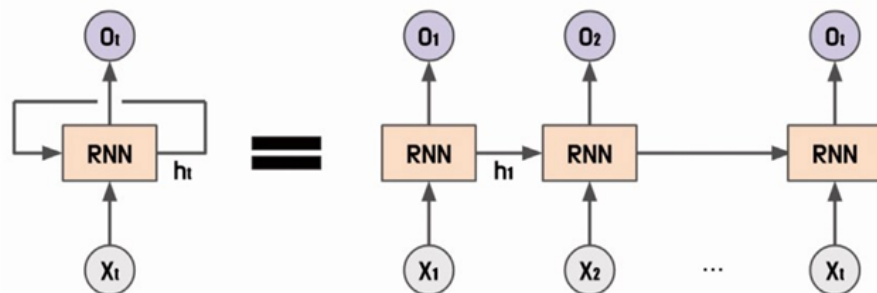


그림 6.12 순환 신경망의 셀

- 순환 신경망은 각 시점 t 에서 현재 입력값 x_i 와 이전 시점 $t - 1$ 의 은닉 상태 h_{i-1} 를 이용해 현재 시점의 은닉 상태 h_i 와 출력값 y_i 를 계산

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$
$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- σ_h 는 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
- 이전 시점 $t - 1$ 의 은닉 상태 h_{t-1} 과 입력값 x_t 를 입력받아 현재 시점의 은닉 상태 h_t 를 계산
 - 이때 σ_h 는 가중치(W)와 편향(b)를 이용해 계산
 - W_{hh} 는 이전 시점의 은닉 상태 h_{t-1} 에 대한 가중치, W_{xh} 는 입력값 x_t 에 대한 가중치, b_h 는 은닉 상태 h_t 의 편향 의미

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$
$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- σ_y 는 순환 신경망의 출력값을 계산하기 위한 활성화 함수
- 현재 시점의 은닉 상태 h_t 를 입력으로 받아 출력값 y_t 를 계산
 - 가중치(W)와 편향(b)를 이용해 계산
 - W_{hy} 는 현재 시점의 은닉 상태 h_t 에 대한 가중치, b_y 는 출력값 y_t 의 편향 의미
- 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터를 처리 가능
- 단순 순환 구조부터 일대다 구조, 다대일 구조, 다대다 구조 등이 있음

일대다 구조

- **일대다 구조(One-to-Many)**
 - 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조
 - 일대다 구조를 사용하여 문장을 입력으로 받고, 문장에서 각 단어의 품사를 예측하는 작업 할 수 있음
 - 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 **이미지 캡셔닝 (Image Captioning)** 모델이 됨

- 입력 시퀀스는 이미지, 출력값은 이미지에 대한 설명 문장들
 - 일대다 구조를 구현하기 위해서는 출력 시퀀스의 길이 미리 알고 있어야 함
- ⇒ 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이 예측하는 모델 함께 구현

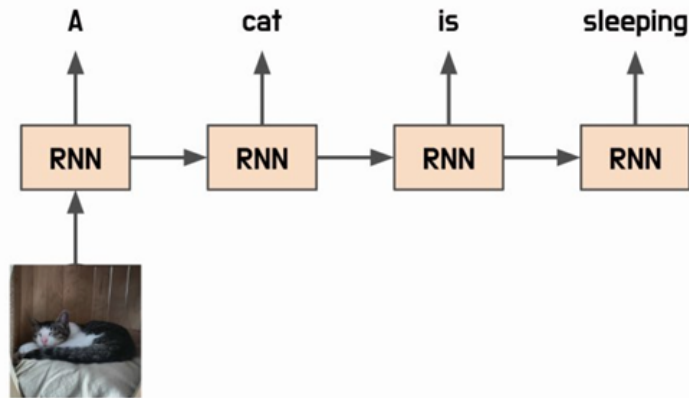


그림 6.13 순환 신경망의 일대다 구조

다대일 구조

• 다대일 구조(Many-to-One)

- 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조
- **감성 분류(Sentiment Analysis)** 분야에서는 다대일 구조 사용하여 특정 문장의 감정(긍정, 부정) 예측하는 작업을 할 수 있음
 - 입력 시퀀스는 문장, 출력값은 해당 문장의 감정(긍정, 부정)
- 입력 시퀀스가 어떤 범주에 속하는지는 구분하는 문장 분류, 두 문장 간의 관계를 추론하는 **자연어 추론(Natural Language Inference)** 등에도 적용

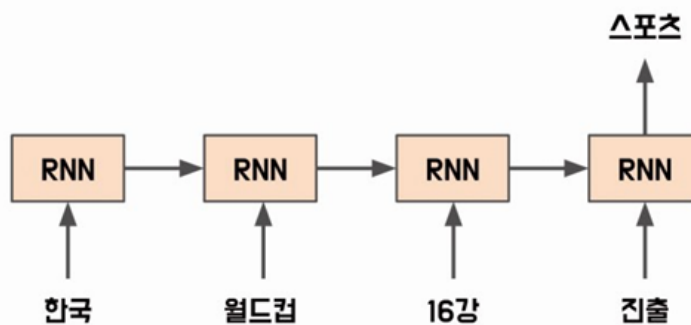


그림 6.14 순환 신경망의 다대일 구조

다대다 구조

- 다대다 구조(Many-to-Many)

- 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조
- 번역기, 음성 인식기 등에서 사용
- 입력 시퀀스와 출력 시퀀스의 길이가 서로 다른 경우 패딩을 추가하거나 잘라내는 등 전처리 과정이 수행됨

- 시퀀스-시퀀스(Seq2Seq) 구조로 이뤄짐

- 입력 시퀀스를 처리하는 **인코더(Encoder)**, 출력 시퀀스를 생성하는 **디코더(Decoder)**로 구성
- 인코더는 입력 시퀀스를 처리해 고정 크기의 벡터를 출력, 디코더는 이 벡터를 입력으로 받아 출력 시퀀스를 생성

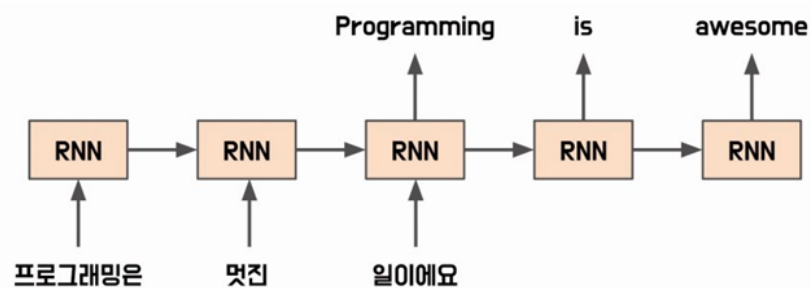


그림 6.15 순환 신경망의 다대다 구조

양방향 순환 신경망

- 양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN)

- 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식
- 이전 시점($t - 1$)의 은닉 상태뿐 아니라 이후 시점($t + 1$)의 은닉 상태도 함께 이용
- t 시점 이후의 데이터도 t 시점의 데이터를 예측하는데 사용
→ 입력 데이터를 순방향으로 처리하는 것만이 아닌, 역방향으로 거꾸로 읽어 들여 처리하는 방식
- 양방향적 정보를 모두 고려하여 현재 시점의 출력값을 계산

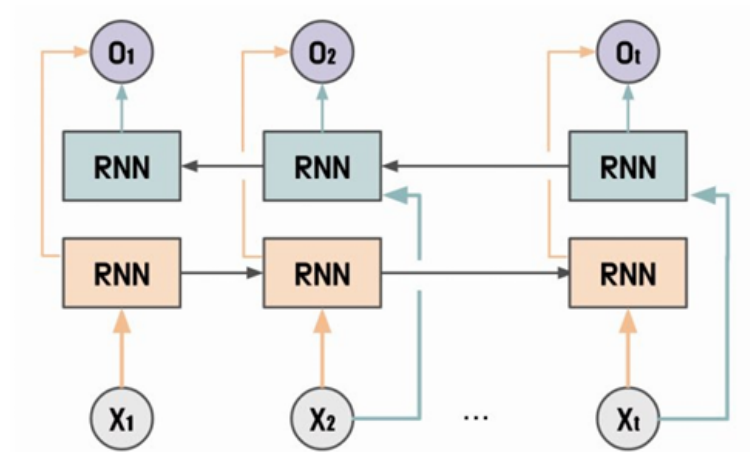


그림 6.16 양방향 순환 신경망 구조

다중 순환 신경망

• 다중 순환 신경망(Stacked Recurrent Neural Network)

- 여러 개의 순환 신경망을 연결하여 구성한 모델로, 각 순환 신경망이 서로 다른 정보를 처리하도록 설계되어 있음
- 여러 개의 순환 신경망 층으로 구성되며, 각 층에서의 출력값은 다음 층으로 전달되어 처리됨
- 데이터의 다양한 특징 추출할 수 있어 성능 향상, 층이 깊어질수록 더 복잡한 패턴 학습 가능
- 층이 많아질수록 학습 시간 오래 걸리고 기울기 소실 문제 발생할 가능성 높아지는 단점

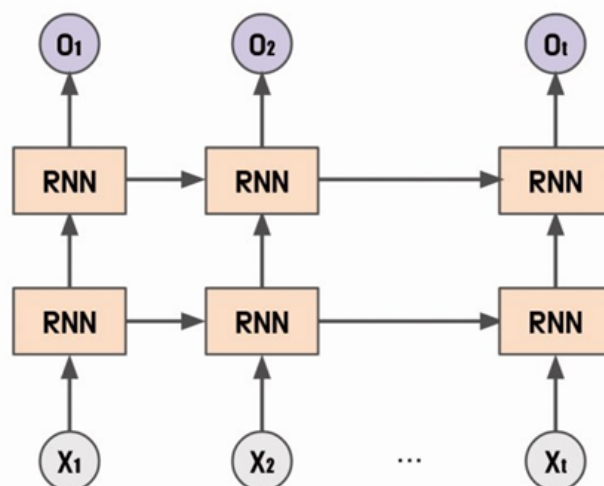


그림 6.17 다중 순환 신경망 구조

순환 신경망 클래스

- 파이토치는 순환 신경망 클래스를 제공



```
#순환 신경망 클래스
rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1,
    nonlinearity="tanh",
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False
)
```

- 순환 신경망 클래스(**RNN**)
 - 입력 특성 크기(**input_size**)와 은닉 상태 크기(**hidden_size**) 입력해 순환 신경망을 구성
 - 입력 특성은 입력값 x , 은닉 상태는 h 를 의미
 - 계층 수(**num_layers**)
 - 순환 신경망의 층수
 - 2 이상이면 다중 순환 신경망을 구성
 - 활성화 함수(**nonlinearity**)
 - 순환 신경망에서 사용되는 활성화 함수 σ 의 종류를 설정
 - **tanh**, **relu** 적용 가능
 - 편향(**bias**)
 - 편향 값 사용 유무를 설정
 - 배치 우선(**batch_first**)
 - 입력 배치 크기를 첫 번째 차원으로 사용할지 여부 설정
 - 참 값이면 [배치 크기, 시퀀스 길이, 입력 특성 크기] 형태로 입력
 - 거짓 값이면 [시퀀스 길이, 배치 크기, 입력 특성 크기]로 입력
 - 드롭아웃(**dropout**)
 - 과대적합 방지를 위한 드롭아웃 확률 설정
 - 양방향 순환 신경망(**bidirectional**)

- 순환 신경망 구조가 양방향으로 처리할지 여부 설정



#양방향 다층 신경망

```
import torch
from torch import nn

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, hidden = model(inputs, h_0)
print(outputs.shape)
print(hidden.shape)

torch.Size([4, 6, 512])
torch.Size([6, 4, 256])
```

- 순환 신경망 클래스는 순전파 연산 시 입력값(`input`)과 초기 은닉 상태(`h_0`)로 순방향 연산을 수행해 출력값(`outputs`)과 최종 은닉 상태(`hidden`)를 반환
 - 입력값 차원은 [배치 크기, 시퀀스 길이, 입력 특성 크기]로 전달
 - 초기 은닉 상태는 순방향 계층 수와 양방향 구조에 따라 전달해야 하는 크기 달라짐
 - [계층 수 x 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]로 구성
 - 순방향 연산 결과는 출력값과 최종 은닉 상태가 반환됨
 - 출력값은 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) x 은닉 상태 크기]가 됨
 - 최종 은닉 상태는 초기 은닉 상태와 동일한 차원으로 반환됨

- 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의

장단기 메모리

• 장단기 메모리(Long Short-Term Memory, LSTM)

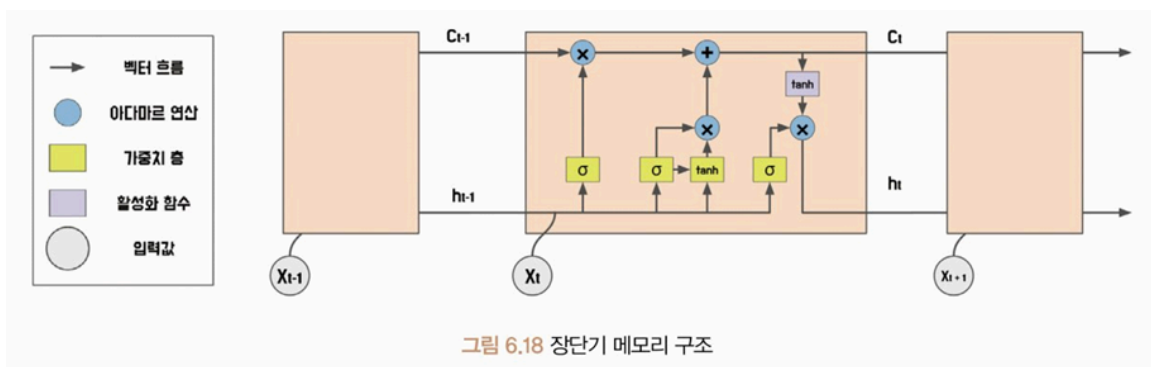
- 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제 해결한 모델
- 일반적인 순환 신경망은 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값을 예측하는 구조이므로 시간적으로 먼 과거의 정보는 잘 기억하지 못함
 - 시간적으로 연속된 데이터를 다룰 수 있단 장점이 있지만, 앞선 시점에서의 정보를 끊임없이 반영하기에 학습 데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달되지 않는다는 단점 있음

→ 장기 의존성 문제(Long-term dependencies) 발생 가능

- 활성화 함수(하이퍼볼릭 탄젠트, ReLU)의 특성으로 인해 역전파 과정에서 기울기 소실, 기울기 폭주 발생 가능

⇒ 이러한 문제 해결 위해 장단기 메모리 사용

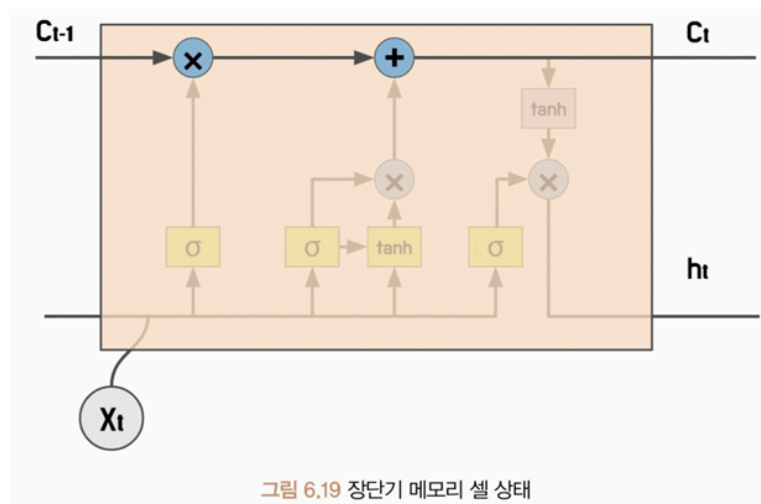
- 장단기 메모리는 **메모리 셀(Memory cell)**과 **게이트(Gate)**라는 구조 도입해 장기 의존성 문제와 기울기 소실 문제를 해결



- 장단기 메모리는 **셀 상태(Cell state)**와 **망각 게이트(Forget gate)**, **기억 게이트(Input gate)**, **출력 게이트(Output gate)**로 정보의 흐름 제어
 - 셀 상태
 - 정보 저장하고 유지하는 메모리 역할
 - 출력 게이트와 망각 게이트에 의해 제어

- 망각 게이트
 - 장단기 메모리에서 이전 셀 상태에서 어떠한 정보를 삭제할지 결정하는 역할
 - 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 삭제할지 결정
- 입력 게이트
 - 새로운 정보를 어떤 부분에 추가할지 결정하는 역할
 - 현재 입력과 이전 셀 상태를 입력으로 받아 어떤 정보를 추가할지 결정
- 출력 게이트
 - 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할
 - 현재 입력과 이전 셀 상태, 새로 추가된 정보를 입력으로 받아 출력할 정보를 결정

장단기 메모리 구조



- 메모리 셀은 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장하는 역할 함
 - 출력값 계산에 직접 사용되지는 않음
- 장단기 메모리는 망각 게이트, 입력 게이트, 출력 게이트를 통해 어떤 정보를 버릴지, 어떤 정보를 기억할지, 어떤 정보를 출력할지 결정하는 추가적인 연산을 수행
 - 세 가지 게이트 모두 활성화 함수로 시그모이드 사용
 - 게이트의 출력값은 0~1 사이의 값으로, 입력값에 대해 얼마나 많은 정보 통과시킬지 결정
- 망각 게이트

- 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수 거친 값과 메모리 셀을 곱한 값으로 이전 시점의 메모리 셀 업데이트
- 어떤 정보를 유지할 것인지, 망각할 것인지 결정

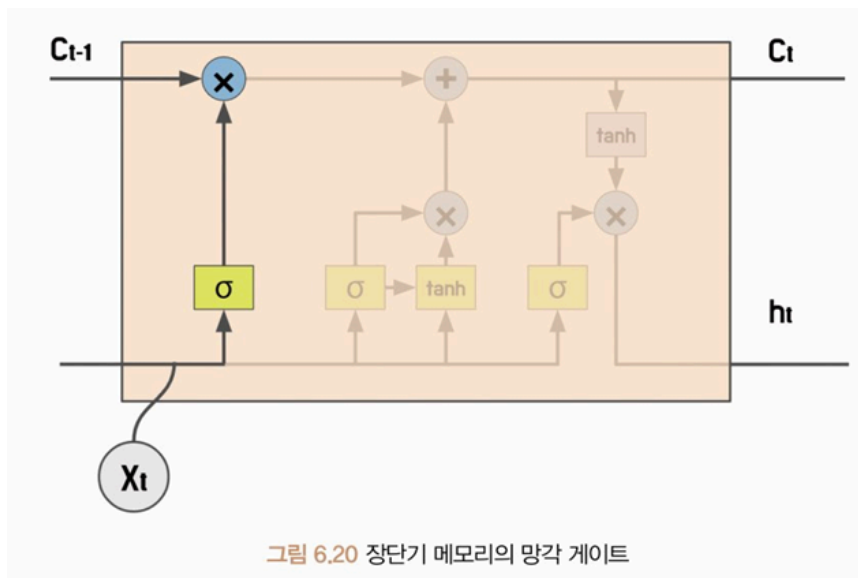
• 기억 게이트

- 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 하이퍼볼릭 탄젠트 함수를 거친 값의 곱으로 새로운 기억 값 계산
- 어떤 정보를 얼마나 추가할지 결정

• 출력 게이트

- 현재 시점의 입력과 이전 시점의 은닉 상태, 새로 업데이트된 메모리 셀을 입력으로 받아 현재 시점의 출력값을 계산
- 어떤 정보를 출력할지를 결정

⇒ 입력값에 대한 가중치를 동적으로 조절하면서, 적절한 정보만을 기억하고 유지할 수 있다는 장점



- 망각 게이트는 이전 시점의 은닉상태 h_{i-1} 과 현재 시점의 입력값 x_i 통해 연산 수행

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

- 출력값은 0~1 사이의 값으로, 0에 가까우면 더 많은 정보를 삭제

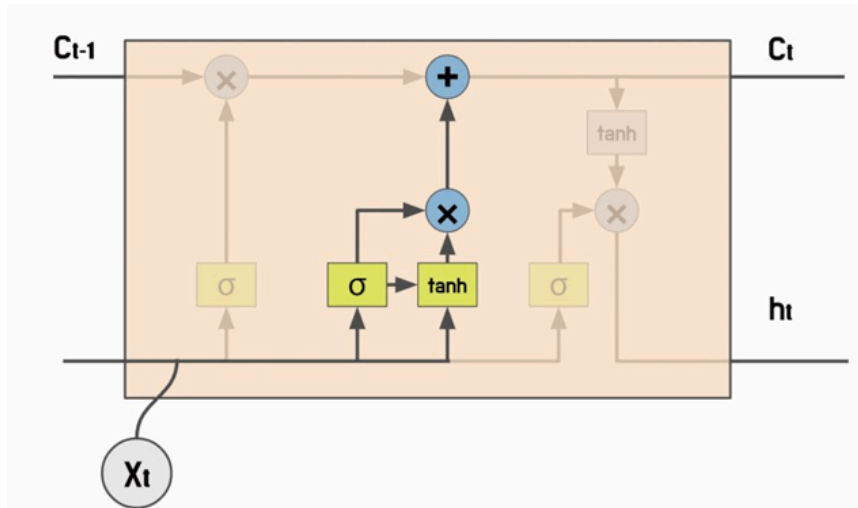


그림 6.21 장단기 메모리의 기억 게이트

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)} x_t + W_x^{(g)} h_{t-1} + b^{(g)})$$

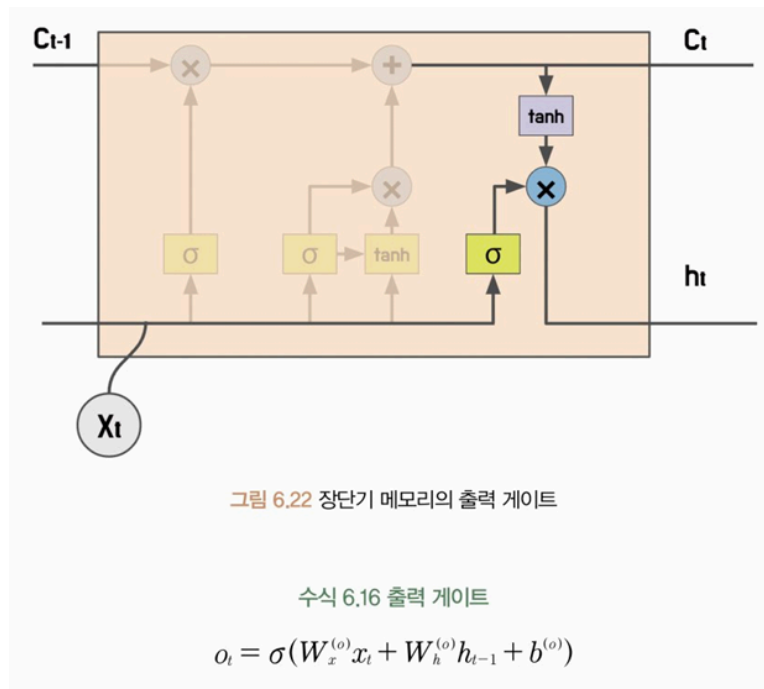
$$i_i = \text{sigmoid}(W_x^{(i)} x_t + W_x^{(i)} h_{t-1} + b^{(i)})$$

- 기억 게이트는 g_i, i_t 로 구성되어 있으며 망각 게이트와 동일한 연산으로 계산됨
 - i_t 가 0에 가까울수록 정보를 망각하게 됨
- 메모리 셀은 망각 게이트와 기억 게이트의 정보로 현재 시점의 메모리 셀 값을 계산

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- 망각 게이트 출력값 f_t 와 이전 시점의 메모리 셀 값 c_{t-1} 을 **아다마르 곱(Hadamard Product, 원소별 곱셈 연산 의미)**하면 현재 시점의 메모리 값이 계산됨
 - 망각 게이트 값 f_t 가 0에 가까울수록 이전 시점의 메모리 셀 값이 현재 시점의 메모리 셀 값에 영향 미치지 않게 됨
 - 기억 게이트도 아다마르 곱을 통해 계산되며 망각 게이트와 합산됨
 - 망각 게이트는 이전 시점의 메모리 셀을 얼마나 유지할지, 기억 게이트는 현재 시점의 새로운 정보를 얼마나 받아들일지를 결정



- 출력 게이트는 현재 시점의 은닉 상태를 제어
- 출력 게이트와 현재 메모리 셀을 연산하면 새로운 은닉 상태 계산할 수 있음

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

- 두 값을 아다마르 곱 연산하면 현재 시점의 은닉 상태가 이전 시점의 은닉 상태에서 얼마나 영향을 받는지를 계산할 수 있게 됨
- 이러한 방법으로 장단기 메모리는 현재 은닉 상태에서 이전 은닉 상태의 정보의 중요도를 제어할 수 있음

장단기 메모리 클래스

- 파이토치에서 장단기 메모리 클래스를 제공



```
#장단기 메모리 클래스
lstm = torch.nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size=0
)
```

- 장단기 메모리 클래스는 활성화 함수를 명확하게 정의해 사용하므로 활성화 함수 매개 변수는 사용 X
- **투사 크기(`proj_size`)** 매개변수 사용해 장단기 메모리 계층의 출력에 대한 **선형 투사(Linear Projection)** 크기를 결정
 - 투사 크기가 0보다 큰 경우, 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑
 - 이 값을 통해 출력 차원을 줄이거나 다른 차원으로 변환 가능
 - 은닉 상태 크기(`hidden_size`)보다 작은 값으로 설정



#양방향 다층 장단기 메모리

```
import torch
from torch import nn

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True
proj_size = 64

model = nn.LSTM(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    batch_first=True,
    bidirectional=bidirectional,
    proj_size=proj_size,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else ouput_size,
)
c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, (h_n, c_n)=model(inputs, (h_0,c_0))

print(outputs.shape)
print(h_n.shape)
print(c_n.shape)
```

```
torch.Size([4, 6, 128])
torch.Size([6, 4, 64])
torch.Size([6, 4, 256])
```

- 장단기 메모리 클래스는 순전파 연산 시 입력값(`inputs`)과 초기 은닉 상태(`h_0`), 초기 메모리 셀 상태(`c_0`)로 순방향 연산을 수행해 출력값(`outputs`)과 최종 은닉 상태(`h_n`), 최종 메모리 셀 상태(`c_n`)를 반환
 - 입력값 차원은 [배치 크기, 시퀀스 길이, 입력 특성 크기]를 전달
 - 초기 은닉 상태도 순환 신경망 클래스와 동일한 구조, 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력
 - 초기 메모리 셀은 [계층 수 × 양방향 여부 + 1, 배치 크기, 은닉 상태 크기], 선형 투사를 적용하더라도 은닉 상태 크기를 사용

- 순방향 연산 결과는 출력값과 최종 은닉 상태, 최종 메모리 셀 상태가 반환
 - 출력값은 수식도 선형 투사가 적용됐으므로 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) × 투사 크기(또는 은닉 상태 크기)]로 구성
 - 최종 은닉 상태는 초기 은닉 상태와 동일한 차원 형태로 반환
- 장단기 메모리는 순환 신경망과 비슷한 구조를 갖지만, 투사 크기가 적용되는 경우 차원 계산 방식이 달라짐
- 순환 신경망 입출력 차원은 배치 우선 매개변수에 따라 차원의 형태가 달라지므로 매개변수 설정에 주의해야 함
- 메모리 셀의 상태도 입력으로 전달해야 하며, 은닉 상태와 메모리 셀 상태를 튜플로 묶어 사용하므로 사용에 주의

모델 실습

- 순환 신경망과 장단기 메모리 활용해 문장 긍/부정 분류 모델 학습

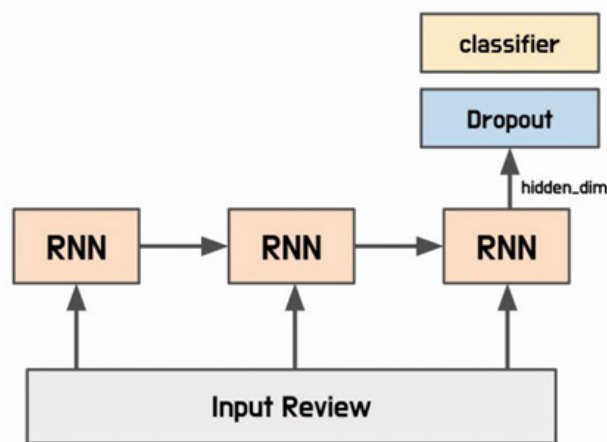


그림 6.23 긍/부정 분류 모델 구조

- 임베딩은 임베딩 벡터를 가져오는 임베딩 계층을 의미
 - 임베딩 계층은 [어휘 사전의 크기 x 임베딩 벡터의 크기]로 순람표 구조 가짐
 - 입력 텍스트를 정수 인코딩한 뒤 해당 색인 값에 해당하는 임베딩을 가져오는 역할
 - 초깃값으로는 무작위 값을 할당, 학습을 통해 임베딩 계층이 최적화되게 만들거나 사전학습된 임베딩 벡터 가져와 사용 가능



#문장 분류 모델

from torch import nn

```
class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm"
    ):
        super().__init__()

        self.embedding = nn.Embedding(
            num_embeddings=n_vocab,
            embedding_dim=embedding_dim,
            padding_idx=0
        )
        if model_type == "rnn":
            self.model = nn.RNN(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        elif model_type == "lstm":
            self.model = nn.LSTM(
                input_size=embedding_dim,
                hidden_size=hidden_dim,
                num_layers=n_layers,
                bidirectional=bidirectional,
                dropout=dropout,
                batch_first=True,
            )

        if bidirectional:
            self.classifier = nn.Linear(hidden_dim * 2, 1)
        else:
            self.classifier = nn.Linear(hidden_dim, 1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        output, _ = self.model(embeddings)
        last_output = output[:, -1, :]
        last_output = self.dropout(last_output)
        logits = self.classifier(last_output)
        return logits
```

- `SentenceClassifier` 클래스
 - 임베딩 층을 구성할 때 사용하는 단어 사전 크기(`n_vocab`)와 순환 신경망 클래스와 장단기 메모리 클래스에서 사용하는 매개변수를 입력으로 전달 받음
 - 모델 종류(`model_type`) 매개변수로 순환 신경망을 사용할지, 장단기 메모리를 사용할지 설정
- 초기화 메서드에서는 `SentenceClassifier` 클래스에 입력된 함수의 매개변수에 따라 모델 구조를 미세 조정
 - 분류기 계층은 모델을 양방향으로 구성한다면 전달되는 입력 채널 수가 달라지므로 분류기 계층을 현재 모델 구조에 맞게 변경
- 순방향 메서드에서는 입력받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값을 얻음
 - 얻은 임베딩 값을 모델에 입력하여 출력값을 얻음
 - 출력값(`output`)의 마지막 시점만 활용할 예정이므로 `[:, -1, :]` 으로 마지막 시점의 결괏값만 분리해 분류기 계층에 전달



```
#데이터세트 불러오기
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")
corpus_df = pd.DataFrame(corpus.test)

train = corpus_df.sample(frac=0.9, random_state=42)
test = corpus_df.drop(train.index)

print(train.head(5).to_markdown())
print("Training Data Size :", len(train))
print("Testing Data Size :", len(test))
```

	text	label
33553	모든 편견을 날려 버리는 가슴 따뜻한 영화. 로버트 드 니로, 필립 세이모어 호프만 영원하라.	1
9427	무한 리메이크의 소재. 감독의 역량은 항상 그 자리에...	0
199	신날 것 없는 애니.	0
12447	간간 격동	1
39489	오랜만에 찾은 주말의 영화의 보석	1

Training Data Size : 45000
Testing Data Size : 5000



#데이터 토큰화 및 단어 사전 구축

```
from konlpy.tag import Okt
from collections import Counter

def build_vocab(corpus, n_vocab, special_tokens):
    counter = Counter()
    for tokens in corpus:
        counter.update(tokens)
    vocab = special_tokens
    for token, count in counter.most_common(n_vocab):
        vocab.append(token)
    return vocab

tokenizer = Okt()
train_tokens = [tokenizer.morphs(review) for review in train.text]
test_tokens = [tokenizer.morphs(review) for review in test.text]

vocab = build_vocab(corpus=train_tokens, n_vocab=5000, special_tokens=["<pad>", "<unk>"])
token_to_id = {token: idx for idx, token in enumerate(vocab)}
id_to_token = {idx: token for idx, token in enumerate(vocab)}

print(vocab[:10])
print(len(vocab))
```

```
['<pad>', '<unk>', '.', '이', '영화', '의', '..', '가', '에', '...']
5002
```

- **Okt** 토큰라이저를 사용해 단어를 토큰화하고 단어 사전을 구축한다
 - 문장의 길이를 맞추기 위해 **<pad>** 토큰을 **special_tokens**에 추가
- 예제에서는 단어 사전의 최대 길이를 5000으로 입력, 특수 토큰이 현재 2개이므로 단어 사전은 5002개



#정수 인코딩 및 패딩

```
import numpy as np

def pad_sequences(sequences, max_length, pad_value):
    result = list()
    for sequence in sequences:
        sequence = sequence[:max_length]
        pad_length = max_length - len(sequence)
        padded_sequence = sequence + [pad_value] * pad_length
        result.append(padded_sequence)
    return np.asarray(result)

unk_id = token_to_id["<unk>"]
train_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in train_tokens
]
test_ids = [
    [token_to_id.get(token, unk_id) for token in review] for review in test_tokens
]

max_length = 32
pad_id = token_to_id["<pad>"]
train_ids = pad_sequences(train_ids, max_length, pad_id)
test_ids = pad_sequences(test_ids, max_length, pad_id)

print(train_ids[0])
print(test_ids[0])
```

```
[ 223 1716   10 4036 2095  193  755   4   2 2330 1031  220  26  13
 4839   1   1   1   2   0   0   0   0   0   0   0   0   0
   0   0   0   0]
[3307   5 1997  456   8   1 1013 3906   5   1   1  13 223  51
   3   1 4684   6   0   0   0   0   0   0   0   0   0
   0   0   0   0]
```

- 학습 데이터셋과 테스트 데이터셋을 단어 사전에 있는 정수로 인코딩
 - 정수로 인코딩하면 모델이 텍스트 데이터를 처리하기 쉬워짐
- 최대 길이(`max_length`)는 입력 텍스트의 길이를 고려해 결정
 - 너무 큰 경우 입력 행렬의 크기가 커져 많은 리소스를 필요로 하게 되며, 너무 짧은 경우 문장을 제대로 반영하지 못해 모델의 성능이 저하될 수 있음
- `pad_sequences` 함수
 - 너무 긴 문장은 최대 길이로 줄이고, 너무 작은 길이라면 최대 길이와 동일한 크기로 변환
 - 시퀀스를 최대 길이로 잘라내고, 시퀀스 길이가 작으면 `<pad>` 토큰을 시퀀스 뒤에 이어 붙여 동일한 길이로 변경

- OOV의 경우 1(`<unk>`)로 인코딩되며, 시퀀스 길이가 짧은 경우 최대 길이 (32)가 될 수 있게 0(`<pad>`) 토큰이 추가됨



#데이터로더 적용

```
import torch
from torch.utils.data import TensorDataset, DataLoader

train_ids = torch.tensor(train_ids)
test_ids = torch.tensor(test_ids)

train_labels = torch.tensor(train.label.values, dtype=torch.float32)
test_labels = torch.tensor(test.label.values, dtype=torch.float32)

train_dataset = TensorDataset(train_ids, train_labels)
test_dataset = TensorDataset(test_ids, test_labels)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
```

- 텐서 데이터세트(`TensorDataset`) 클래스는 파이토치 텐서 형태를 입력값으로 받으므로 정수 인코딩과 라벨값을 파이토치 텐서 형태로 변환
 - 변환된 데이터세트는 데이터로더 클래스에 적용



#손실 함수와 최적화 함수 정의

```
from torch import optim
```

```
n_vocab = len(token_to_id)
```

```
hidden_dim = 64
```

```
embedding_dim = 128
```

```
n_layers = 2
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
classifier = SentenceClassifier(
```

```
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim, n_layers=n_layers  
) .to(device)
```

```
criterion = nn.BCEWithLogitsLoss().to(device)
```

```
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)
```

- 학습에 적용하려는 하이퍼파라미터를 선언
 - 은닉 상태 크기는 64, 임베딩 벡터의 크기는 128로 적용하고 신경망을 두 개의 층으로 구성
- 문장의 긍/부정을 분류하므로 손실 함수는 이진 교차 엔트로피 함수를 적용
 - `BCEWithLogitsLoss` 클래스는 `BCELoss` 클래스와 `Sigmoid` 클래스가 결합된 형태
 - `BCELoss` 클래스를 사용하는 경우, `criterion(torch.sigmoid(output), target)` 과 동일한 형태로 간주할 수 있음
- 최적화 함수는 **RMSProp(Root Mean Square Propagation)**을 적용
 - RMSProp은 모든 기울기를 누적하지 않고, **지수 가중 이동 평균 (Exponentially Weighted Moving Average, EWMA)**을 사용해 학습률을 조절
 - ⇒ 기울기 제곱 값의 평균값이 작아지면 학습률을 증가시키고, 그 반대일 경우 학습률을 감소시켜서 불필요한 지역 최솟값에 빠지는 것을 방지
 - RMSprop은 기울기의 크기가 큰 경우에는 빠른 수렴을 보이며, 작은 경우에는 더 작은 학습률을 유지함으로써 더 안정적으로 최적화를 수행



```
#모델 학습 및 테스트
def train(model, datasets, criterion, optimizer, device, interval):
    model.train()
    losses = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if step % interval == 0:
            print(f"Train Loss {step}: {np.mean(losses)}")

def test(model, datasets, criterion, device):
    model.eval()
    losses = list()
    corrects = list()

    for step, (input_ids, labels) in enumerate(datasets):
        input_ids = input_ids.to(device)
        labels = labels.to(device).unsqueeze(1)

        logits = model(input_ids)
        loss = criterion(logits, labels)
        losses.append(loss.item())
        yhat = torch.sigmoid(logits)>.5
        corrects.extend(

        torch.eq(yhat, labels).cpu().tolist()
        )

    print(f"Val Loss : {np.mean(losses)}, Val Accuracy : {np.mean(corrects)}")

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)

    Train Loss 0: 0.6808339357376099
    Train Loss 500: 0.6943095798739892
    Train Loss 1000: 0.6913632521857986
    Train Loss 1500: 0.6830024296724344
    Train Loss 2000: 0.6755679306657477
    Train Loss 2500: 0.6587954610049939
    Val Loss : 0.5416280304471525, Val Accuracy : 0.7338
```

```
Train Loss 0: 0.2212703675031662
Train Loss 500: 0.2816699924181678
Train Loss 1000: 0.290330159970067
Train Loss 1500: 0.2931845419972559
Train Loss 2000: 0.29459350064642603
Train Loss 2500: 0.29705547500864976
Val Loss : 0.3906621564262972, Val Accuracy : 0.8238
```

- 에폭마다 테스트 데이터셋으로 모델의 **검증 손실(Validation Loss)**과 **검증 정확도(Validation Accuracy)** 확인
- 출력 결과를 보면 테스트 데이터셋에 대해서도 손실이 감소하며, 정확도가 상승하는 것을 확인할 수 있음
- 모델 학습 과정에서 임베딩 계층을 비롯한 순환 신경망 내의 여러 가중치가 최적화됨
- 학습된 임베딩 계층의 가중치를 동일한 단어 사전을 사용하는 토큰의 임베딩 값으로 사용할 수도 있음



#학습된 모델로부터 임베딩 추출

```
token_to_embedding = dict()
```

```
embedding_matrix = classifier.embedding.weight.detach().cpu().numpy()
```

```
for word, emb in zip(vocab, embedding_matrix):
```

```
    token_to_embedding[word] = emb
```

```
token = vocab[1000]
```

```
print(token, token_to_embedding[token])
```

```
보고싶다 [-0.79336643  0.6709532  0.9988691 -0.4406818  2.4278314 -0.5132188
-1.6413862  0.4102169  1.3801705 -0.02338726  0.6745436  1.2422966
-0.14099254 -0.58963645 -0.73427325  0.05959708 -0.44999588 -0.5615407
-0.05955222 -0.40623587 -1.4944198 -2.0459132  0.4367292 -0.69952774
-0.8717556  0.8465241  0.42154264  0.9938335  0.7133947  0.00575377
 0.4367158 -0.40835017  0.9121726  1.7850952 -0.4019251  0.73861307
-0.21739696  0.1606418  0.5687655  1.4216948 -1.2795627  0.03777466
-1.0840906  0.5129847  0.16543943 -0.04786262  0.355846  0.03168814
-1.9735525 -0.8883228 -0.6192509 -0.20703971  0.74091583 -2.9069808
 0.07006218  1.327318 -0.03431427  0.16519837  0.49238515  1.4596015
 1.4773252  1.4662119  2.1660788 -0.313472 -0.6613132 -1.1263446
 0.43993226 -1.3961322  0.11335947  1.9552312 -1.0449558 -0.6115691
 1.2952724 -0.44639596 -1.156955  0.39141643  0.2016408  0.9641113
-1.486294 -0.701918  1.0437994  1.734434  2.2221508  1.5576879
-0.2197506 -1.4808869 -0.72538674  0.68134624 -0.3376854 -0.50074106
 1.7189022 -0.35173732 -1.9415973 -0.28951392  0.78087455 -1.3486516
 0.6698399  1.3643713 -0.34456208 -0.069527  1.6252809  0.9811217
-1.2251712 -0.8940953 -0.14237265  0.99470484 -0.7961068 -0.7479401
 0.45642042  1.2877783 -0.6180284  0.64995235 -0.2388691 -1.4915586
 0.06549539  0.86025304  0.71313524 -0.310035 -0.27538672  0.8306695
-0.83146346 -1.1716632 -0.4125453  0.33005136 -0.93452984 -0.7704845
 0.37611774 -0.9435285 ]
```

- 학습된 모델의 임베딩 계층을 각 단어에 대한 임베딩으로 사용할 수 있음
- 하지만 긍/부정 분류는 임베딩 계층이 아닌 순환 신경망의 연산이 더 중요하게 동작
- 모델이 복잡할수록 임베딩 계층이 토큰의 의미 정보를 학습하기 어려움
- 사전 학습된 임베딩 값을 초깃값으로 적용해 모델 학습



```
#사전 학습된 모델로 임베딩 계층 초기화
from gensim.models import Word2Vec

word2vec = Word2Vec.load("/content/models/word2vec.model")
init_embeddings = np.zeros((n_vocab, embedding_dim))

for index, token in id_to_token.items():
    if token not in ["<pad>", "<unk>"]:
        init_embeddings[index] = word2vec.wv[token]

embedding_layer = nn.Embedding.from_pretrained(
    torch.tensor(init_embeddings, dtype=torch.float32)
)
```

- 사전 학습된 모델로 임베딩 계층을 초기화 시 넘파이 배열로 초기화 가능
 - 이때 `<pad>` 토큰과 `<unk>` 토큰은 초기화에서 제외한다.
- 임베딩 계층은 파이토치의 임베딩 클래스의 `from_pretrained` 메서드로 초기화할 수 있음
 - 이 값을 `SentenceClassifier` 클래스의 `self.embedding` 값으로 적용한다.



```
#사전 학습된 임베딩 계층 적용
class SentenceClassifier(nn.Module):
    def __init__(
        self,
        n_vocab,
        hidden_dim,
        embedding_dim,
        n_layers,
        dropout=0.5,
        bidirectional=True,
        model_type="lstm",
        pretrained_embedding=None
    ):

        if pretrained_embedding is not None:
            self.embedding = nn.Embedding.from_pretrained(
                torch.tensor(pretrained_embedding, dtype=torch.float32)
            )
        else:
            self.embedding = nn.Embedding(
                num_embeddings=n_vocab,
                embedding_dim=embedding_dim,
                padding_idx=0
            )
```

- 사전 학습된 임베딩(`pretrained_embedding`) 매개변수가 `None` 이 아니라면 전달된 값을 임베딩 계층으로 초기화



#사전 학습된 임베딩을 사용한 모델 학습

```
classifier = SentenceClassifier(
    n_vocab=n_vocab, hidden_dim=hidden_dim, embedding_dim=embedding_dim,
    n_layers=n_layers, pretrained_embedding=init_embeddings
).to(device)
criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.RMSprop(classifier.parameters(), lr=0.001)

epochs = 5
interval = 500

for epoch in range(epochs):
    train(classifier, train_loader, criterion, optimizer, device, interval)
    test(classifier, test_loader, criterion, device)
```

```
Train Loss 0: 0.6954693794250488
Train Loss 500: 0.66273173802865
Train Loss 1000: 0.5926802434793839
Train Loss 1500: 0.5648595857806876
Train Loss 2000: 0.5443567214296318
Train Loss 2500: 0.5305356110771243
Val Loss : 0.5099662863217984, Val Accuracy : 0.7278
```

```
-----
Train Loss 0: 0.5051383376121521
Train Loss 500: 0.4206651306437875
Train Loss 1000: 0.4211636180733586
Train Loss 1500: 0.42134050378634247
Train Loss 2000: 0.4226531938634355
Train Loss 2500: 0.42219828748431315
Val Loss : 0.41367277812462644, Val Accuracy : 0.8084
```

- 사전 학습된 임베딩을 사용하는 것은 모델 성능을 개선할 수 있는 방법 중 하나이지만 학습 데이터의 양이 충분히 많다면, 모델의 목적에 맞게 새로운 임베딩 층을 학습하는 것이 더 좋은 결과를 얻을 수도 있음
- 사전 학습된 임베딩을 사용하더라도 해당 언어와 문제에 맞는 임베딩을 선택하는 것이 중요

합성곱 신경망

- 합성곱 신경망(Convolutional Neural Network, CNN)
 - 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류

- 입력 데이터의 지역적인 특징을 추출하는 데 특화된 구조를 갖고 있으며 이를 위해 **합성곱(Convolution) 연산**을 사용
- 합성곱 연산은 이미지의 특정 영역에서 입력값의 분포 또는 변화량을 계산해 출력 노드를 생성
 - 특정 영역 안에서 연산을 수행하므로 **지역 특징(Local Features)**을 효과적으로 추출
- 이미지 데이터는 고정된 프레임 내에 객체들의 위치와 형태가 자유분방하므로 여러 영역의 지역 특징을 조합해 입력 데이터의 전반적인 **전역 특징(Global Features)**을 파악 가능
- 2014년 사전 학습된 임베딩과 합성곱 신경망을 이용한 문장 분류 모델의 등장으로 자연어 처리에서도 사용되기 시작
 - ⇒ 입력 데이터의 길이에 상관없이 병렬 처리가 가능하고, 학습에 필요한 가중치 수를 줄여 깊은 신경망을 구성할 수 있게 됨

합성곱 계층

- 입력 데이터와 필터를 합성곱 해 출력 데이터를 생성하는 계층
- 이미지나 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용
- 필터를 사용해 데이터의 특징을 추출하므로 데이터의 지역적인 패턴을 인식할 수 있음
- 입력 데이터의 모든 위치에서 동일한 필터를 사용하므로 모델 매개변수를 공유
 - 모델의 매개변수를 공유함으로써 모델이 학습해야 할 매개변수 수가 감소해 과대적합을 방지
 - 입력 데이터에서 특징을 추출할 때, 해당 특징이 이미지 내 다른 위치에 존재하더라도 필터를 사용해 특징을 추출하므로 특징이 어디에 있어도 동일하게 추출 가능
- 합성곱 계층을 여러 겹 쌓아 모델을 구성하며, 합성곱 계층이 많아질수록 모델의 복잡도가 증가하므로 더 다양한 특징을 추출해 학습할 수 있음

필터

- 합성곱 계층은 입력 데이터에 **필터(Filter)**를 이용해 합성곱 연산을 수행하는 계층
- 필터는 **커널(Kernel)** 또는 **윈도(Window)**로 불리기도 함
- 일반적으로 3x3, 5x5와 같은 작은 크기의 정방향으로 구성
- 필터를 일정 간격으로 이동하면서 입력 데이터와 합성곱 연산을 수행해 특징 맵을 생성
 - 필터 영역마다 합성곱 연산 수행, 필터의 가중치가 이 과정에서 갱신됨



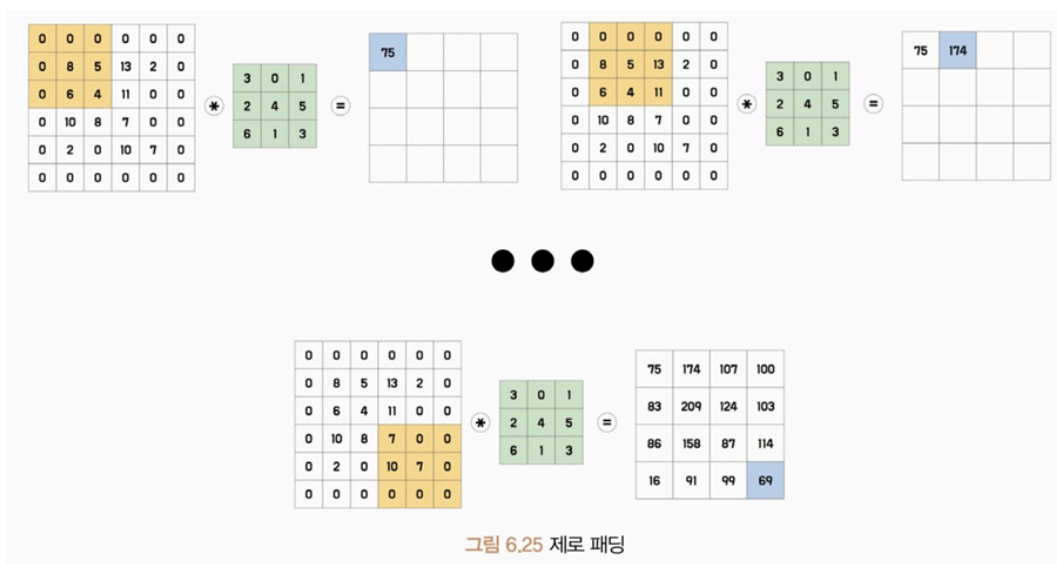
- 각 계층에서 하나의 필터가 여러 번 사용되고 이를 공용 가중치로 공유함으로써 이미지 내에서 어느 위치에서도 동일한 패턴을 학습 가능

패딩

- 일반적으로 합성곱 연산을 수행하면 출력값인 특징 맵의 크기가 작아짐
 - 합성곱 신경망을 더 깊게 쌓는 데 제약, 합성곱 신경망 성능에 악영향 끼칠 수 있음
 - 이미지의 가장자리에 있는 정보는 다른 위치에 있는 정보에 비해 학습 어려움

⇒ 이러한 현상 방지 위해 **패딩(Padding)** 추가

- 입력 이미지나 입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙이는 것
- 제로 패딩: 가장자리에 덧붙이는 패딩 값을 0으로 할당



- 합성곱 신경망에서는 패딩을 적절히 사용해 입력 데이터의 크기를 조정, 작은 크기의 필터로도 이미지 전체에 대한 정보를 반영

간격

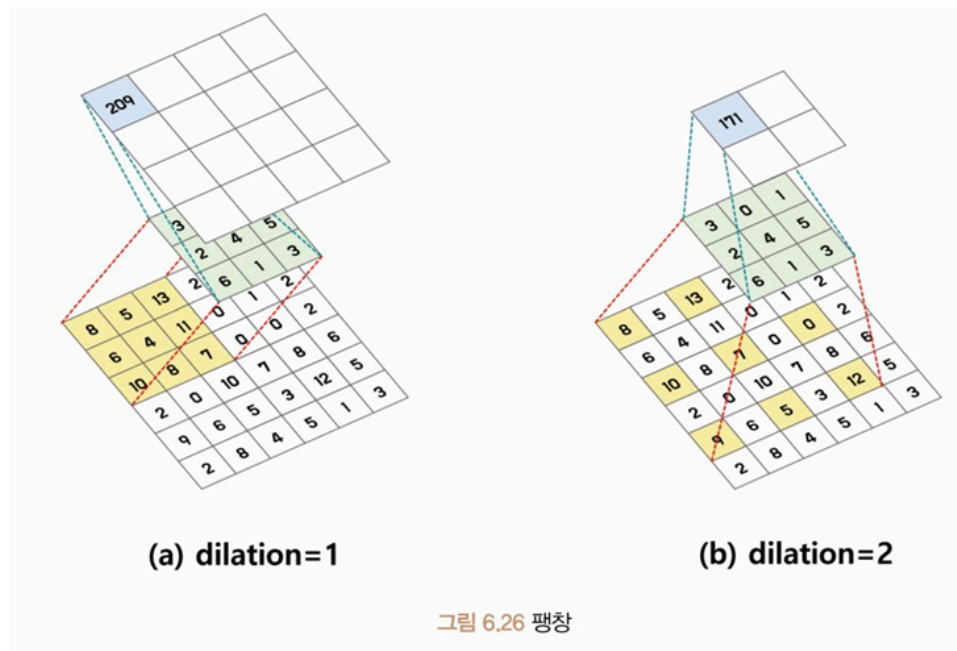
- **간격(Stride)**는 필터가 한 번에 움직이는 크기를 의미
- 간격의 크기를 조절함으로써 출력 데이터의 크기를 조절 가능
 - 입력 데이터의 공간적인 정보 유지하거나 감소 가능
 - 입력 데이터의 공간적인 정보는 픽셀 간의 상대적인 위치나 거리에 대한 정보를 의미
- 간격을 조절해 합성곱 신경망이 학습해야 하는 모델 매개변수의 수 감소시킬 수 있음
→ 모델의 복잡도 낮추고 과대적합 방지 가능

채널

- **채널(Channel)**에서 입력 데이터와 필터 간의 연산을 수행
- 입력 데이터와 필터가 4차원으로 구성되어 있을 때 같은 위치의 값끼리 연산되게 함
→ 입력 데이터의 공간 정보를 유지하면서 추출되는 특징 확장 가능
- 채널 개수는 일반적으로 합성곱 계층에서 설정, 모델의 구조나 목적에 따라 달라짐
 - 개수 많아질수록 학습할 수 있는 특징의 다양성이 증가해 모델의 **표현력 (Representational Power)**이 높아지는 효과
- 출력 채널이 많은 경우 각 채널은 입력 데이터에서 서로 다른 특징을 학습
 - 더 많은 종류의 특징 학습, 더 복잡한 문제를 해결할 수 있는 능력 갖추게 됨
 - 모델의 매개변수 많아지므로 학습 시간과 메모리 사용량이 증가하는 단점, 모델 과대적합의 위험

팽창

- **팽창(Dilation)**은 합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법
 - 필터와 입력 데이터 사이에 간격을 두는 방법
 - 값이 커질수록 필터가 입력 데이터를 바라보는 범위 넓어짐
→ 입력 데이터의 각 픽셀이 출력에 미치는 영향이 강화되는 효과



- 팽창은 필터의 크기를 키우지 않고 입력 데이터에 더 넓은 영역을 고려할 수 있게 해 더 깊고 복잡한 모델을 구성할 수 있게 함
 - 팽창을 적절히 적용하면 모델의 매개변수 수를 줄일 수 있어 메모리 사용량 줄일 수 있음
 - 필터가 바라봐야 하는 입력 데이터의 범위가 커지므로 오히려 연산량이 늘어날 수 있음
 - 팽창 크기가 너무 크다면 인접한 픽셀값을 고려하지 않게 되므로 공간적인 정보가 보존되지 않아 특징 추출의 효과가 떨어질 수 있음

합성곱 계층 클래스

- 파이토치에서 합성곱 계층 클래스 사용해 신경망 구현 가능



#2차원 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(  
    in_channels,  
    out_channels,  
    kernel_size,  
    stride=1,  
    padding=0,  
    dilation=1,  
    groups=1,  
    bias=True,  
    padding_mode="zeros"  
)
```

- 합성곱 계층 클래스는 **입력 데이터 채널 크기(`in_channels`)**와 **출력 데이터 채널 크기(`out_channels`)**를 통해 채널을 생성
- **커널 크기(`kernel_size`)** 합성곱 연산의 필터 크기 의미, **간격(`stride`)**, **패딩(`padding`)**, **팽창(`dilation`)**은 필터의 속성 의미
- **그룹(`group`)**
 - 입력 채널과 출력 채널을 하나의 그룹으로 묶는 것 의미
 - 2 이상의 값으로 설정 시 입력 채널과 출력 채널을 더 작은 그룹으로 나눠 각각 합성곱 연산을 수행
 - 합성곱 연산 시 모델의 매개변수 수를 줄일 수 있음
 - 입력 채널이나 출력 채널이 그룹 수의 배수여야 함
- **편향 설정(`bias`)**
 - 계층에 편향 값 포함 여부 설정
- **패딩 모드(`padding_mode`)**
 - 패딩 영역에 할당되는 값을 설정
 - **zeros** : 제로 패딩 의미
 - **reflect** : 입력 데이터의 가장자리를 거울처럼 반사해 값을 할당
 - **replicate** : 가장자리의 값을 복사해 값을 할당
- 합성곱 계층은 필터를 통해 입력 데이터의 공간적인 정보를 유지하거나 감소시켜 출력 크기를 합성곱 계층에서 제어할 수 있음

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$
$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

- 이미지의 픽셀 위치는 소수점으로 할당할 수 없으므로 정수형으로 변환

활성화 맵

- **활성화 맵(Activation Map)**
- 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값을 의미
 - 합성곱 계층에서 입력 이미지와 필터의 합성곱 연산을 통해 특징 맵을 추출하면 이 값에 비선형성을 추가하기 위해 활성화 함수를 적용
 - 일반적으로 ReLU 함수 적용
 - 특징 맵의 값이 0보다 크면 그대로, 0 이하인 경우 0을 출력
 - 얻은 활성화 맵은 다음 계층의 입력값으로 사용
- 합성곱 연산과 활성화 함수를 여러 번 반복하여 신경망을 구성하면 입력 이미지에서 추출된 추상적인 특징을 학습할 수 있게 됨
- 합성곱 계층의 출력값에 활성화 함수 적용하지 않으면 합성곱 연산 결과값이 그대로 다음 계층으로 전달됨
 - 이 경우 모델이 선형적인 결합만 수행하게 되어 복잡한 패턴, 추상적 특징 학습하는 것이 어려워짐
- ⇒ 활성화 함수를 적용함으로써 모델이 비선형성을 가지게 되어 입력 데이터에서 다양한 추상적인 특징을 학습할 수 있게 됨
- 활성화 맵은 합성곱 모델을 분석하는 데 매우 중요한 정보를 제공
 - 활성화 맵 시각화하여 확인하면 입력 이미지에서 모델이 어떤 특징을 학습하는지, 어떤 부분이 활성화되어 있는지 등 이해 가능

풀링

- 풀링(Pooling)

- 특징 맵의 크기를 줄이는 연산으로, 합성곱 계층 다음에 적용됨
- 특징 맵의 크기를 줄여 연산량을 감소시키고 입력 데이터의 정보를 압축하는 효과

- 최댓값 풀링(Max Pooling)

- 특정 크기의 필터 내 원숫값 중 가장 큰 값을 선택해 특징 맵의 크기 감소



그림 6.27 최댓값 풀링

- 평균값 풀링(Average Pooling)

- 필터 내 원숫값의 평균값으로 특징 맵의 크기 감소

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

- 풀링은 입력 데이터의 공간적 크기를 줄여 계산 비용 감소 가능, 입력 데이터의 특징 위치가 변경되어도 인근 영역에 대한 연산을 적용하기 때문에 공간적 정보 유지 가능
- 입력 데이터의 위치 정보를 일부 손실시키기 때문에 세밀한 위치 정보가 필요한 작업에서는 정능 저하 초래 가능
 - 따라서 최근에는 연산량이 더 많더라도 합성곱 계층의 간격을 설정해 입력 데이터의 공간적인 크기를 줄이는 방법 사용

풀링 클래스



```
#2차원 최대값 풀링 클래스
pool = torch.nn.MaxPool2d(
    kernel_size,
    stride=None,
    padding=0,
    dilation=1
)
```

- 특징 맵 내의 최대값 선택하는 방법이므로 패딩 모드 적용 X

```
#2차원 평균값 풀링 클래스
pool= torch.nn.AvgPool2d(
    kernel_size,
    stride=None,
    padding=0,
    count_include_pad=True
)
```

- 평균값으로 풀링해야 하므로 팽창 지원 X
 - 팽창의 경우 원소 간의 거리가 멀어지므로 주변 영역의 특징 계산 어려워짐
- 패딩 포함(`count_include_pad`)
 - 패딩 영역의 값을 평균 계산에 포함할지 여부 설정
 - 참값으로 사용하면 제로 패딩의 값이 평균값 풀링 연산에 포함됨

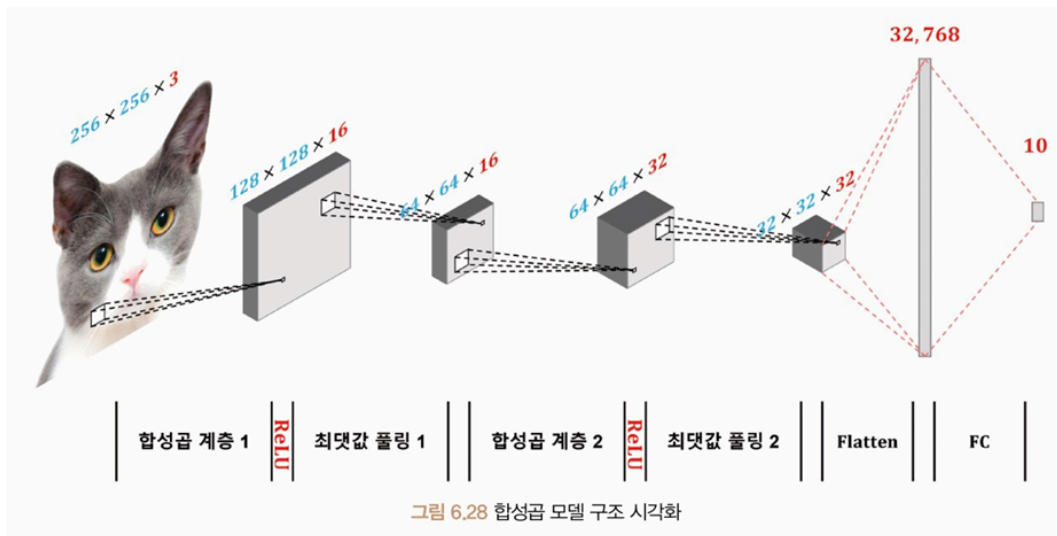
완전 연결 계층

- 완전 연결 계층(Fully Connected Layer, FC)
 - 각 입력 노드가 모든 출력 노드와 연결된 상태 의미
 - 완전 연결 계층은 입력과 출력 간의 모든 가능한 관계를 학습 가능
 - 출력 노드의 수를 조절할 수 있으므로 모델의 복잡성과 용량 조절에 사용
- 합성곱 신경망에서는 합성곱 계층과 풀링 계층을 거친 결과물인 특징 맵을 입력으로 받음
 - 입력된 3차원 특징 맵은 평탄화 작업 통해 1차원 벡터로 변경, 완전 연결 계층의 가중치와 내적 연산을 수행해 출력값을 계산

→ 이전 계층에서 추출한 특성 맵의 공간 정보가 무시되고 모든 입력을 독립적으로 처리해 계산

- 합성곱 신경망에서는 특성 맵의 공간 정보를 보존하기 위해 합성곱 계층으로 구성된 네트워크를 구성, 이후에 완전 연결 계층을 추가하여 고수준 작업을 수행
- 완전 연결 계층은 이전 계층에서 추출된 특징을 활용하여 최종적인 분류 작업을 수행
 - 최종 값은 소프트맥스, 시그모이드와 같은 활성화 함수를 적용해 분류 모델로 구성 가능

⇒ 완전 연결 계층은 합성곱 신경망의 최종 출력을 결정하는 역할을 함



- 입력 이미지 → 합성곱 계층, ReLU, 최대값 풀링을 2번 반복하고 3차원 배열을 1차원 벡터로 펼치는 **flatten** 연산 수행, 이 1차원 벡터를 완전 연결 계층으로 전달해 10개의 출력 벡터 반환
 - 10개의 출력 벡터에 소프트맥스나 시그모이드 적용 → 이미지를 입력했을 때 클래스를 분류하는 모델 구축 가능



```
#합성곱 모델
import torch
from torch import nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Sequential(
            nn.Conv2d(
                in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.conv2 = nn.Sequential(
            nn.Conv2d(
                in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1
            ),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )

        self.fc = nn.Linear(32 * 32 * 32, 10)

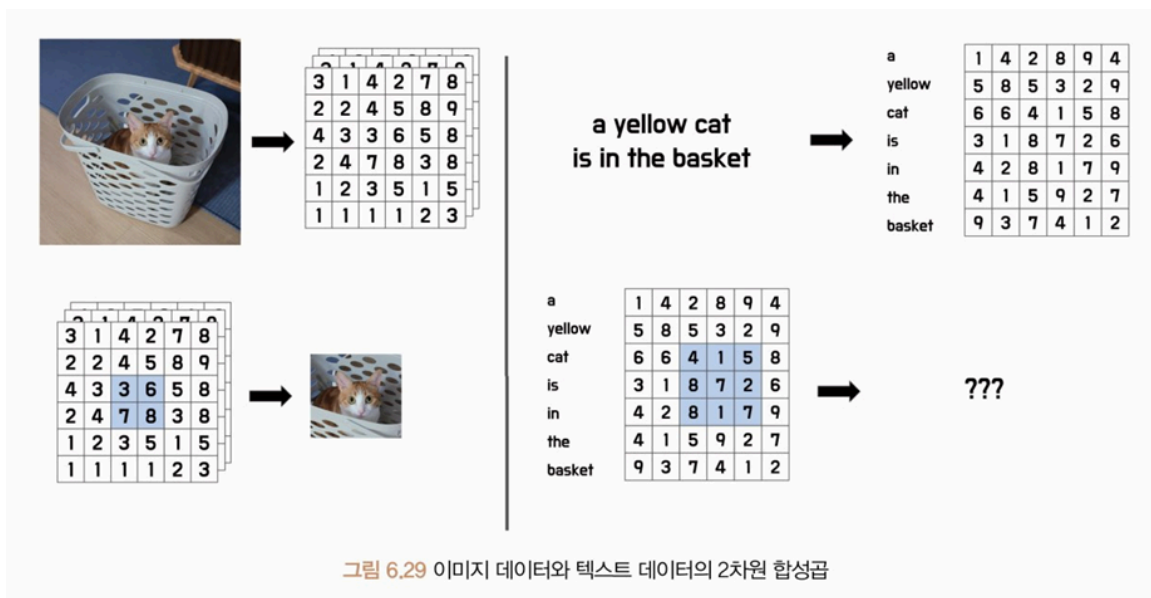
    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = torch.flatten(x)
        x = self.fc(x)
        return x
```

- 완전 연결 계층은 선형 변환으로 구성, 입력 벡터와 가중치의 곱으로 계산됨
- 입력 데이터 차원 크기는 마지막 특징 맵의 높이 x 너비 x 채널
- 출력 데이터 차원의 크기가 분류 모델의 클래스 개수 됨

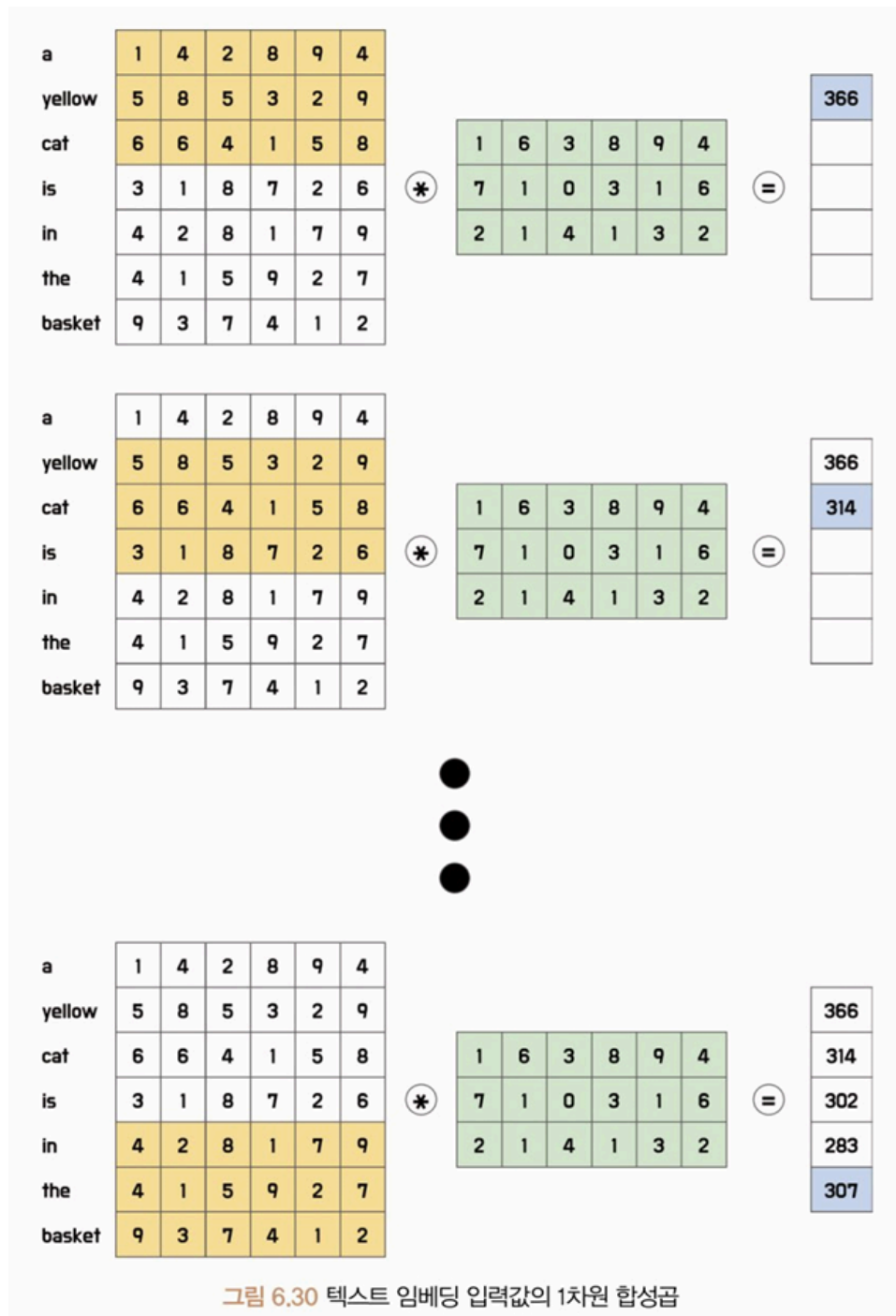
모델 실습

- 합성곱 신경망을 활용해 자연어 처리 모델 구성
- **2차원 합성곱(2-Demensional Convolution)**
 - 수직/수평 방향으로 이동하며 합성곱 연산을 수행
- 텍스트 데이터의 임베딩 값은 입력 순서를 제외하면 입력값의 위치가 의미 가지지 않음

→ 이미지 데이터와 같이 2차원 합성곱 필터를 사용하면 텍스트의 정보 제대로 학습할 수 없음



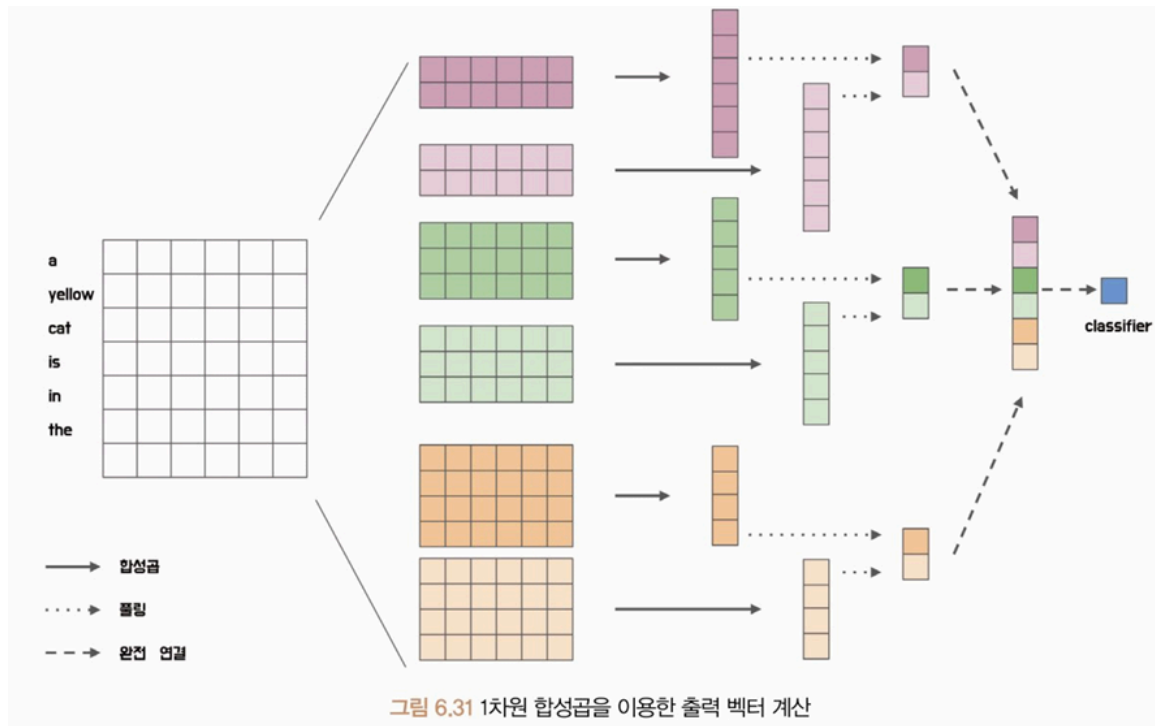
- 텍스트 데이터는 **1차원 합성곱(1-Dimensional Convolution)** 적용해야 함
- 1차원 합성곱은 텍스트 데이터 처리에서 많이 사용
 - 텍스트 데이터는 문장을 단어 단위로 분리하여 각 단어를 임베딩하여 나온 1차원 벡터 데이터를 입력값으로 사용
 - 이러한 입력 값에 수직 방향으로만 이동하는 필터를 적용하여 해당 문장의 특징을 추출
 - 1차원 필터의 크기는 필터의 높이에만 영향 미치며, 필터의 너비는 입력 임베딩의 크기 됨



- 필터 크기가 3인 합성곱 임베딩을 수행하면 인접한 3개의 토큰에 대해 연산 수행
 - N-gram과 유사한 개념, 텍스트 순서에 따른 정보 학습 가능
- 1차원 합성곱을 이용한 신경망에서는 다양한 크기의 합성곱 필터를 사용하여 여러 종류의 정보 추출 가능
 - 1차원 합성곱을 수행하면 1차원 벡터를 출력값으로 얻게 되므로 풀링을 적용하면 하나의 스칼라값이 도출됨

→ 크기가 다른 여러 개의 합성곱 필터 사용하면 여러 개의 스칼라값 얻을 수 있음
 → 이런 다양한 크기의 출력 벡터를 모아 하나의 벡터로 연결하여 하나의 특징 벡터로 만들 수 있음

- 특징 벡터를 이용해 분류, 예측 등의 작업 수행 가능



- 합성곱 신경망을 이용한 문장 분류는 일반적으로 사전 학습된 임베딩 벡터 사용



#합성곱 기반 문장 분류 모델 정의

```
import torch
from torch import nn

class SentenceClassifier(nn.Module):
    def __init__(self, pretrained_embedding, filter_sizes, max_length, dropout=0.5):
        super().__init__()

        self.embedding = nn.Embedding.from_pretrained(
            torch.tensor(pretrained_embedding, dtype=torch.float32)
        )
        embedding_dim = self.embedding.weight.shape[1]

        conv = []
        for size in filter_sizes:
            conv.append(
                nn.Sequential(
                    nn.Conv1d(
                        in_channels=embedding_dim,
                        out_channels=1,
                        kernel_size=size
                    ),
                    nn.ReLU(),
                    nn.MaxPool1d(kernel_size=max_length-size-1),
                )
            )
        self.conv_filters = nn.ModuleList(conv)

        output_size = len(filter_sizes)
        self.pre_classifier = nn.Linear(output_size, output_size)
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(output_size, 1)

    def forward(self, inputs):
        embeddings = self.embedding(inputs)
        embeddings = embeddings.permute(0, 2, 1)

        conv_outputs = [conv(embeddings) for conv in self.conv_filters]
        concat_outputs = torch.cat([conv.squeeze(-1) for conv in conv_outputs], dim=1)

        logits = self.pre_classifier(concat_outputs)
        logits = self.dropout(logits)
        logits = self.classifier(logits)
        return logits
```

- **SentenceClassifier** 클래스

- 사전 학습된 임베딩 벡터(`pretrained_embedding`), 필터 크기 리스트(`filter_sizes`), 최대 길이(`max_length`)를 입력받아 모델 구성
- 합성곱 계층 구성 시 크기가 다른 여러 개의 합성곱 필터를 구성하기 위해 필터 크기 리스트만큼 시퀀셜 생성

- 각 시퀀스는 1차원 합성곱 계층, ReLU, 최댓값 풀링으로 구성
- 합성곱 계층의 출력 채널 수는 1, 최댓값 풀링의 커널 크기는 앞선 합성곱 연산의 영향을 받으므로 최대 길이 - 필터 크기 - 1로 설정
- `ModuleList` 클래스
 - 여러 개의 서브 모듈을 리스트 형태로 저장하는 기능을 제공
 - `conv` 리스트에는 시퀀스 모듈이 담겨 있으므로 `ModuleList` 클래스를 적용
- 순방향 메서드에서는 입력값(`inputs`)을 임베딩 계층에 통과시켜 임베딩 벡터를 얻음
- 이후 `permute` 메서드를 통해 차원을 변경하고 앞서 설정한 여러 개의 합성곱 필터에 통과시킴
- `cat` 메서드를 통해 각 필터의 출력값을 이어 붙여 하나의 벡터로 만들고 분류를 위한 네트워크를 통과시켜 최종 출력값을 얻음



```
#합성곱 신경망 분류 모델 학습
device = "cuda" if torch.cuda.is_available() else "cpu"
filter_sizes = [3, 3, 4, 4, 5, 5]
classifier = SentenceClassifier(
    pretrained_embedding=init_embeddings,
    filter_sizes=filter_sizes,
    max_length=max_length
).to(device)

criterion = nn.BCEWithLogitsLoss().to(device)
optimizer = optim.Adam(classifier.parameters(), lr=0.001)
```

```
Train Loss 0: 0.697555422782898
Train Loss 500: 0.5926512637419139
Train Loss 1000: 0.5566606014877648
Train Loss 1500: 0.5363474553680515
Train Loss 2000: 0.5238535415152202
Train Loss 2500: 0.5172986053898543
Val Loss : 0.4636144690905897, Val Accuracy : 0.7806
```

```
Train Loss 0: 0.3722414970397949
Train Loss 500: 0.46787036889922357
Train Loss 1000: 0.46270977675200226
Train Loss 1500: 0.46545651733756144
Train Loss 2000: 0.46371746115360424
Train Loss 2500: 0.4651123295422317
Val Loss : 0.44566648751021193, Val Accuracy : 0.7904
```

- 필터 크기 변수의 값, 개수 조절해 모델의 구조 변경하거나 확장 가능
- 최적화 함수로 **Adam(Adaptive Moment Estimation)** 사용
 - 경사 하강법 알고리즘을 개선한 방법, 모멘텀과 RMSProp을 결합
 - 이전 기울기 값의 지수 이동 평균과 이전 기울기 값의 지수 이동 제곱 평균을 사용해 현재 기울기 값을 갱신
 - 지난 기울기 값의 영향을 일정 부분 유지하면서 새로운 기울기 값을 반영하기 때문에 기울기가 빠르게 변화하는 구간에서 더 안정적인 수렴 (모멘텀)
 - 학습률을 조절해 발산하는 경우 방지 (RMSProp)
- Adam은 학습률, 모멘텀, RMSProp의 하이퍼파라미터를 모두 자동으로 조절하면서 최적의 값으로 수렴
 - 하이퍼파라미터 조정하는 수고 줄어듦, 학습률과 모멘텀의 조절 문제 해결
 - SGD보다 더 빠른 수렴 보이는 경우 많음

- 텍스트 데이터를 합성곱 신경망으로 처리하면 각 단어를 지역적인 특징으로 인식할 수 있어 문장 내부의 구조적인 정보를 잘 파악 가능
- 합성곱 신경망은 계산 속도 빠르기 때문에 대규모 데이터셋에서도 빠르게 학습 가능
- 합성곱 신경망은 이미지와 유사한 구조를 가진 2차원 데이터에 더 적합하므로 텍스트 데이터를 합성곱 신경망으로 처리한다면 모델의 구조 및 목적을 충분히 고려해야 함